Learned Spatiotemporal Region Proposal for Subpixel Target Detection in Large-Scale Overhead Persistent Surveillance Video

Stage 1 Architecture Specification

Architecture Design Document

March 2026

Abstract

This document specifies a two-stage, fully learned pipeline for detecting and segmenting small moving targets in large spatiotemporal volumes characteristic of overhead persistent surveillance. The operational scenario involves a ground sample distance (GSD) under 100 m with targets whose physical extent is well below the pixel scale, yielding single-pixel or subpixel apparent target signatures. The scene volume is $1024 \times 1024 \times 64$ (spatial $\times$ spatial $\times$ temporal frames). Detection is fundamentally a spatiotemporal discrimination problem: no single frame contains sufficient information to distinguish targets from background clutter, which includes multi-layered moving clouds, apparent terrain motion from sensor parallax, and sensor jitter.

A second-stage model—a 3D convolutional U-Net with a 3D Swin Transformer bottleneck operating on $128 \times 128 \times 64$ patches—has been validated as an effective discriminator on small spatial extents. However, it cannot scale to full $1024 \times 1024 \times 64$ volumes due to GPU memory constraints.

This document focuses on Stage 1: a learned region proposal network based on bidirectional state-space models (Mamba) that processes the full $1024 \times 1024 \times 64$ volume at native spatial resolution, producing a multi-channel heatmap encoding target likelihood and coarse motion direction. Connected-component extraction with motion-coherence filtering yields a small set of candidate patches ($\leq$15 per volume) that are forwarded to Stage 2 for precise segmentation. The full pipeline is designed to operate within 192 GB of GPU memory under sequential stage execution.

We present the problem formulation grounded in remote sensing physics, computer vision theory, and machine learning principles, followed by the complete architectural specification with memory analysis and justification against alternative approaches.

Contents

1 Introduction **5**
- 1.1 Scope and Organization . . . 5

2 Problem Formulation **5**
- 2.1 Notation . . . 5
- 2.2 Image Formation Model . . . 6
  - 2.2.1 Subpixel Target Appearance . . . 6
  - 2.2.2 Background Clutter Model . . . 7
- 2.3 The Spatiotemporal Discrimination Problem . . . 7
  - 2.3.1 Why Single-Frame Detection Fails . . . 8
  - 2.3.2 Temporal Discrimination via Motion Incoherence . . . 8
- 2.4 Scale and Computational Constraints . . . 8
- 2.5 Problem Statement . . . 9

3 Solution Overview **10**
- 3.1 What Stage 1 Does . . . 10
- 3.2 Why Two Stages . . . 10
- 3.3 Design Constraints and Their Consequences . . . 10
- 3.4 Processing Pipeline . . . 11
- 3.5 Key Design Insights . . . 12
- 3.6 Post-Processing Summary . . . 13
- 3.7 Training Overview . . . 13

4 Theoretical Foundations **13**
- 4.1 Remote Sensing Theory . . . 14
  - 4.1.1 Point Spread Function and Subpixel Energy Distribution . . . 14
  - 4.1.2 Temporal SNR Integration . . . 14
  - 4.1.3 Apparent Motion from Platform Parallax . . . 15
  - 4.1.4 Atmospheric Attenuation and Cloud Interaction . . . 15
- 4.2 Computer Vision Theory . . . 15
  - 4.2.1 Spatiotemporal Feature Extraction . . . 15
  - 4.2.2 Insufficiency of Spatial-Only Features . . . 16
  - 4.2.3 Motion Coherence as a Discriminative Signal . . . 16
  - 4.2.4 Limitations of Classical Approaches . . . 17
- 4.3 Machine Learning Theory . . . 17
  - 4.3.1 3D Convolutions and the Factorization Problem . . . 17
  - 4.3.2 State-Space Models for Temporal Sequence Modeling . . . 18
  - 4.3.3 Channel Attention and Gating Mechanisms . . . 24
  - 4.3.4 Depthwise Separable Convolutions for Spatial Mixing . . . 25
  - 4.3.5 Loss Functions for Extreme Class Imbalance . . . 25

5 Stage 1 Architecture Specification **26**
- 5.1 Architecture Overview . . . 26
- 5.2 Input Preprocessing: Multi-Scale Temporal Differences . . . 27
- 5.3 Channel Projection . . . 28
- 5.4 Bidirectional Mamba Backbone . . . 29

5.4.1 Spatial Chunking . . . 30
5.4.2 Forward and Backward SSM Passes . . . 31
5.4.3 Bidirectional Fusion . . . 32
5.5 Temporal Aggregation . . . 33
5.6 PSF-Matched Spatial Mixer . . . 34
5.7 Output Heads . . . 35
5.8 Memory Analysis . . . 36
5.8.1 Activation Memory . . . 37
5.8.2 Weight and Optimizer Memory . . . 37
5.8.3 Peak Memory Estimate . . . 37
5.9 Architectural Degrees of Freedom . . . 38

6 Two-Stage Pipeline and Interface **38**
6.1 Pipeline Overview . . . 38
6.2 Post-Processing: From Heatmap to Candidate Patches . . . 38
6.2.1 Detection Thresholding . . . 39
6.2.2 Motion Coherence Filtering . . . 39
6.2.3 Connected Component Extraction and Patch Generation . . . 39
6.3 Patch Overlap and Boundary Handling . . . 39
6.4 Stage 2 Interface Contract . . . 40
6.5 Output Assembly . . . 40
6.6 Evaluation Metrics . . . 40

7 Justification and Alternatives Analysis **41**
7.1 Decision Framework . . . 41
7.2 Alternative 1: Dense 3D CNN at Full Resolution . . . 41
7.3 Alternative 2: (2+1)D Factored Convolutions . . . 42
7.4 Alternative 3: Temporal Shift Module (TSM) . . . 42
7.5 Alternative 4: Spatial Self-Attention (Vision Transformer) . . . 43
7.6 Alternative 5: Per-Pixel Temporal Models Without Spatial Mixing . . . 43
7.7 Alternative 6: Brute-Force Tiling with Stage 2 . . . 44
7.8 Alternative 7: Classical Detect-Before-Track . . . 44
7.9 Alternative 8: Recurrent Feature Refinement (RFR / Deformable Alignment) . . . 45
7.10 Alternative 9: Sparse 3D Convolutions . . . 45
7.11 Summary of Alternatives . . . 46

8 Training Requirements **46**
8.1 Synthetic Data Requirements . . . 46
8.1.1 Scene Composition . . . 46
8.1.2 Occlusion Modeling . . . 47
8.1.3 Ground-Truth Labels . . . 47
8.2 Loss Formulation . . . 48
8.3 Training Curriculum . . . 48
8.4 Data Augmentation . . . 49

9 Operational Considerations **49**
9.1 Full Pipeline Memory Budget 49
9.2 Inference Throughput 49
9.3 Hardware Requirements 50
9.4 Deployment Considerations 50
9.5 Prototyping Roadmap 51

1 Introduction

Overhead persistent surveillance systems provide continuous wide-area observation from elevated platforms, producing video sequences that span large spatial extents over sustained temporal windows. A central mission is the detection and tracking of small, potentially dim targets against complex, dynamic backgrounds.

The challenge addressed in this document arises from a fundamental mismatch between the sensor's ground sample distance and the physical size of the targets of interest. At a GSD under 100 m, targets of interest may be an order of magnitude or more smaller than a single pixel, rendering them subpixel phenomena. Even targets producing sufficient contrast to register on the sensor typically appear as single-pixel or few-pixel brightness perturbations, often below the noise floor in any individual frame.

Detection therefore cannot rely on spatial morphology—the target has no spatial structure at this resolution. Instead, detection requires *spatiotemporal discrimination*: identifying brightness perturbations that evolve over time in a manner distinct from all background motion sources. This requirement, combined with the scale of the data ($1024 \times 1024 \times 64$ volumes), creates a problem that is simultaneously compute-intensive and information-sparse, demanding architectures that preserve spatial resolution while efficiently modeling temporal dynamics.

1.1 Scope and Organization

This document specifies the Stage 1 component of a two-stage detection pipeline:

- **Stage 1** (this document): A learned spatiotemporal region proposal network that processes the full $1024 \times 1024 \times 64$ volume, producing candidate regions of interest with high recall.
- **Stage 2** (existing, validated): A 3D convolutional U-Net with 3D Swin Transformer bottleneck that performs precise segmentation on $128 \times 128 \times 64$ patches extracted from Stage 1 candidates.

The document is organized as follows. Section 2 formalizes the detection problem, including the physical models of target appearance, background clutter, and sensor effects. Section 3 provides a high-level overview of the Stage 1 solution—what it ingests, what it produces, how its processing pipeline is organized, and why the major design decisions follow from the problem constraints. Section 4 presents the theoretical foundations spanning remote sensing, computer vision, and machine learning. Section 5 details the Stage 1 architecture specification. Section 6 describes the full two-stage pipeline and the Stage 1 to Stage 2 interface. Section 7 provides justification against alternative architectures. Section 8 specifies training requirements. Section 9 discusses memory budgets and operational considerations.

2 Problem Formulation

2.1 Notation

We define the notation used throughout this document.

Scene volume. The input is a three-dimensional volume $V \in \mathbb{R}^{H \times W \times T}$, where $H = W = 1024$ are the spatial dimensions and $T = 64$ is the number of temporal frames. Each element $V(i, j, t)$ represents the measured radiance (or a linearly calibrated proxy thereof) at spatial pixel (i, j) and frame t. We treat this as a single-channel signal; extension to multi-band is straightforward.

Spatial coordinates and motion. A spatial position is denoted $\mathbf{p} = (i, j) \in \{1, \ldots, H\} \times \{1, \ldots, W\}$. A velocity vector in pixel coordinates is $\mathbf{v} = (v_i, v_j) \in \mathbb{R}^2$, with units of pixels per frame. The angular direction of motion is $\theta = \text{atan2}(v_j, v_i) \in (-\pi, \pi]$, mapped to $[0, 2\pi)$ by adding 2π to negative values, and the speed is $|\mathbf{v}| = \sqrt{v_i^2 + v_j^2}$.

Targets. A target $k \in \{1, \ldots, K\}$ with $K \in \{0, \ldots, 20\}$ is characterized by a time-varying position $\mathbf{p}_k(t) \in \mathbb{R}^2$ (real-valued, as targets are subpixel), a velocity $\mathbf{v}_k(t)$, and an intrinsic radiance $L_k(t)$. The trajectory of target k is $\mathcal{T}_k = \{\mathbf{p}_k(t)\}_{t=1}^{T}$.

Background components. The noiseless background radiance at pixel $\mathbf{p}$ and frame t is modeled as:

$$B(\mathbf{p}, t) = B_{\text{terr}}(\mathbf{p}, t) + \sum_{l=1}^{N_c} B_{\text{cloud}}^{(l)}(\mathbf{p}, t), \tag{1}$$

where B_{terr} is the apparent terrain radiance (which shifts over time due to sensor platform motion) and $B_{\text{cloud}}^{(l)}$ is the radiance contribution from the l-th cloud layer ($N_c \geq 0$ layers, each translating at an independent velocity $\mathbf{v}_{\text{cloud}}^{(l)}$). Sensor noise $\eta(\mathbf{p}, t)$ is introduced separately in the image formation model (section 2.2).

Point spread function. The sensor's optical point spread function is denoted $h(\mathbf{p})$, typically modeled as a 2D Gaussian with full-width at half-maximum (FWHM) w_{PSF} in pixel units. The corresponding Gaussian standard deviation is $\sigma_{\text{PSF}} = w_{\text{PSF}}/2.355$.

Outputs. Stage 1 produces a heatmap $\mathcal{H} \in \mathbb{R}^{C \times H \times W}$ with $C = 9$ channels: one target likelihood channel and eight directional motion class channels (the batch dimension $B = 1$ is omitted in the notation for clarity but included in architectural specifications). Stage 2 produces a binary segmentation mask $M_{\text{seg}} \in \{0, 1\}^{128 \times 128 \times 64}$ for each candidate patch.

2.2 Image Formation Model

The observed radiance at pixel $\mathbf{p}$ and frame t is:

$$V(\mathbf{p}, t) = \left[B(\mathbf{p}, t) + \sum_{k=1}^{K} S_k^{\text{obs}}(\mathbf{p}, t) \right] * h(\mathbf{p}) + \eta(\mathbf{p}, t), \tag{2}$$

where $*$ denotes 2D spatial convolution with the PSF, $S_k^{\text{obs}}(\mathbf{p}, t)$ is the (potentially cloud-attenuated) radiance contribution of the k-th target at pixel $\mathbf{p}$ and frame t, and $\eta(\mathbf{p}, t)$ is additive sensor noise (read noise, shot noise, and electronic artifacts) introduced after optical integration.

2.2.1 Subpixel Target Appearance

Because target physical extent is well below the GSD, the target is effectively a point source. Its pre-optics radiance contribution is:

$$S_k(\mathbf{p}, t) = L_k(t) \cdot \delta(\mathbf{p} - \mathbf{p}_k(t)), \tag{3}$$

where $\mathbf{p}_k(t) \in \mathbb{R}^2$ is the subpixel target position and δ is the Dirac delta. After convolution with the optical PSF in eq. (2), the apparent target profile on the focal plane becomes $L_k(t) \cdot h(\mathbf{p} - \mathbf{p}_k(t))$—a PSF-shaped spot centered at the subpixel location.

As $\mathbf{p}_k(t)$ shifts continuously, the PSF-weighted energy redistributes fractionally across neighboring pixels. For a Gaussian PSF with $w_{\text{PSF}} \approx 3$–5 pixels, the target's footprint spans a 3×3 to 5×5 pixel neighborhood, with the peak pixel receiving only a fraction of the total target energy.

The *subpixel shift* between frames manifests as a smooth, continuous redistribution of fractional energy across this neighborhood—a signal that is invisible in any single frame but produces a characteristic spatiotemporal signature when viewed across the temporal window.

2.2.2 Background Clutter Model

The background presents three distinct motion regimes, each of which can mimic target signatures in limited observations:

Apparent terrain motion. The sensor platform moves along a trajectory, inducing apparent translation of the terrain. The terrain's apparent velocity on the focal plane, $\mathbf{v}_{\text{terr}}(t)$, is determined by the platform velocity, sensor altitude, and focal length (derived in section 4.1.3); the terrain at frame t appears shifted by $\int_0^t \mathbf{v}_{\text{terr}}(\tau)\, d\tau$. This creates a globally coherent translational motion field that affects all terrain features identically.

Multi-layer cloud motion. Cloud layers at different altitudes translate at independent velocities $\mathbf{v}_{\text{cloud}}^{(l)}$, potentially in different directions. Each layer has a spatially varying optical depth $\tau^{(l)}(\mathbf{p}, t)$ that determines the degree to which it attenuates or obscures the layers below it. Critically, in any single frame, cloud texture is often indistinguishable from terrain texture—both present as spatially varying radiance patterns. Only the temporal evolution (smooth translation at a velocity distinct from terrain) reveals the cloud nature.

A target passing behind a semi-transparent cloud layer with optical depth τ experiences continuous attenuation. The observed (attenuated) target radiance used in eq. (2) is:

$$S_k^{\text{obs}}(\mathbf{p}, t) = S_k(\mathbf{p}, t) \cdot e^{-\tau^{(l)}(\mathbf{p}_k(t), t)}, \tag{4}$$

where S_k is the pre-optics point-source radiance from eq. (3) and $\tau^{(l)}$ is the optical depth of the intervening cloud layer along the line of sight. After PSF convolution in eq. (2), the apparent target profile becomes $L_k(t) \cdot e^{-\tau^{(l)}} \cdot h(\mathbf{p} - \mathbf{p}_k(t))$—the familiar PSF-shaped spot, attenuated by the cloud. For optically thick clouds ($\tau \gg 1$), the target effectively vanishes; for moderate optical depth ($\tau \in [0.3, 0.8]$), the target is attenuated but potentially recoverable. In the clear-sky case, $S_k^{\text{obs}} = S_k$. When multiple cloud layers are present, the total attenuation is the product of per-layer transmissions: $S_k^{\text{obs}}(\mathbf{p}, t) = S_k(\mathbf{p}, t) \cdot \prod_{l=1}^{N_c} e^{-\tau^{(l)}(\mathbf{p}_k(t), t)} = S_k(\mathbf{p}, t) \cdot e^{-\tau_{\text{tot}}(\mathbf{p}_k(t), t)}$, where $\tau_{\text{tot}} = \sum_l \tau^{(l)}$ is the total optical depth used in the single-frame expansion of section 2.3.1.

Sensor jitter. High-frequency perturbations in the sensor pointing introduce frame-to-frame shifts $\boldsymbol{\delta}(t) \sim \mathcal{N}(\mathbf{0}, \sigma_{\text{jitter}}^2 \mathbf{I})$, typically with $\sigma_{\text{jitter}} \in [0.1, 0.3]$ pixels. Jitter affects the entire frame uniformly but creates apparent motion of all stationary scene elements, potentially mimicking or masking subpixel target displacements.

2.3 The Spatiotemporal Discrimination Problem

Given the image formation model of eq. (2), the detection problem is to identify the presence and locations of targets $\{1, \ldots, K\}$ from the volume V.

2.3.1 Why Single-Frame Detection Fails

In any individual frame $V(\cdot, t)$, the contribution of a single target at pixel $\mathbf{p}$ is buried within the PSF-convolved mixture. Expanding eq. (2) for a single target in the neighborhood of $\mathbf{p}_k(t)$:

$$V(\mathbf{p}, t) = \widetilde{B}(\mathbf{p}, t) + L_k(t) \cdot e^{-\tau_{\text{tot}}(\mathbf{p}_k(t), t)} \cdot h(\mathbf{p} - \mathbf{p}_k(t)) + \eta(\mathbf{p}, t), \tag{5}$$

where $\widetilde{B}(\mathbf{p}, t) = [B(\mathbf{p}, t)] * h(\mathbf{p})$ is the PSF-convolved background, $\tau_{\text{tot}}(\mathbf{p}_k(t), t) = \sum_l \tau^{(l)}(\mathbf{p}_k(t), t)$ is the total optical depth from all intervening cloud layers, and the target term reflects both PSF blurring and cloud attenuation (with $\tau_{\text{tot}} = 0$ in the clear-sky case).

The target contribution is a faint perturbation superimposed on the PSF-convolved background $\widetilde{B}(\mathbf{p}, t)$ and noise $\eta(\mathbf{p}, t)$. The instantaneous signal-to-noise ratio at the peak pixel is:

$$\text{SNR}_{\text{frame}}(t) = \frac{L_k(t) \cdot e^{-\tau_{\text{tot}}(\mathbf{p}_k(t), t)} \cdot h(\mathbf{0})}{\sqrt{\text{Var}[\widetilde{B}(\mathbf{p}, t)] + \text{Var}[\eta(\mathbf{p}, t)]}}, \tag{6}$$

which is typically $\text{SNR}_{\text{frame}} < 1$ for the faint-target regime of interest. At this SNR, the target pixel is statistically indistinguishable from background fluctuations in any single frame. No spatial feature—morphology, texture, contrast—can discriminate the target because it occupies fewer pixels than the PSF footprint and has no spatial structure at the sensor resolution.

2.3.2 Temporal Discrimination via Motion Incoherence

Over the full temporal window $T = 64$, a target at velocity $\mathbf{v}_k$ traverses $|\mathbf{v}_k| \cdot T$ pixels, creating a linear trajectory in the (i, j, t) volume. This trajectory is discriminable from background motion sources because:

1. **Terrain motion** is globally coherent: all terrain pixels share the same apparent velocity $\mathbf{v}_{\text{terr}}$ on the focal plane (determined by the platform velocity, altitude, and focal length; see section 4.1.3). A target with $\mathbf{v}_k \neq \mathbf{v}_{\text{terr}}$ produces a residual motion $\mathbf{v}_k - \mathbf{v}_{\text{terr}}$ that is *spatially isolated*—it appears at a single point, not across the entire scene.
2. **Cloud motion** is *regionally* coherent: each cloud layer moves as a spatially extended block at velocity $\mathbf{v}^{(l)}_{\text{cloud}}$. A target that traverses a cloud-free or single-cloud region has a velocity that is *incoherent* with the local background motion field.
3. **Jitter** is globally coherent and temporally uncorrelated: it affects all pixels identically in each frame. A target's smooth, directional displacement is distinguishable from the stochastic, zero-mean jitter signal when observed over sufficient frames.

The discriminative signal is therefore *motion incoherence*: a localized brightness perturbation whose temporal evolution is inconsistent with the spatially coherent motion of all background components at that location. This signal exists only in the joint spatiotemporal domain—it cannot be extracted from any individual frame, any spatial-only analysis, or any temporal-only analysis at a single pixel without spatial context (since a single pixel cannot distinguish target motion direction from cloud motion direction).

2.4 Scale and Computational Constraints

The detection problem is subject to the following constraints that shape the architectural design:

Spatial resolution preservation. Subpixel targets distribute energy across a PSF-sized neighborhood (3×3 to 5×5 pixels). Any spatial downsampling (striding, pooling) destroys the fractional energy gradients that encode target presence and subpixel position. The architecture must process the full 1024×1024 spatial grid at native resolution.

Temporal resolution preservation. A target moving at 0.5 pixels/frame traverses 32 pixels in 64 frames. Temporal downsampling or coarse temporal binning reduces the number of independent measurements of the target's trajectory, degrading the temporal SNR. The architecture should preserve the full 64-frame temporal extent as long as possible in the processing pipeline.

Memory budget. The full volume $V \in \mathbb{R}^{1024\times1024\times64}$ at float32 precision occupies 256 MB. Processing this volume through a neural network that maintains spatial resolution generates intermediate activations proportional to $H \times W \times T \times C$, where C is the channel count. At $C = 16$ and $T = 64$, a single activation tensor occupies $1024 \times 1024 \times 64 \times 16 \times 4$ bytes ≈ 4.3 GB. The total GPU memory budget for the sequential two-stage pipeline is 192 GB, requiring careful control of channel width, temporal reduction strategy, and activation memory management.

Target sparsity. With 0–20 targets per 1024×1024 scene and each target's trajectory spanning at most $\sim$1/5th of the spatial extent (at the maximum velocity of 3 px/frame over 64 frames), the vast majority of the volume contains no target signal. An architecture that commits equal computation to all spatial regions is wasteful; a coarse-to-fine strategy that rapidly identifies regions of interest and concentrates discriminative computation there is strongly preferred.

Generalizability. The system must be fully learned, with no hand-tuned parameters tied to specific sensor characteristics, cloud patterns, or target types. When the sensor, platform, or target population changes, the system should be retrainable on new synthetic or real data without algorithmic modification.

2.5 Problem Statement

Given the above, we define the Stage 1 problem as follows.

> **Stage 1 Problem.** Design a neural network $f_\theta : \mathbb{R}^{H\times W\times T} \to \mathbb{R}^{C\times H\times W}$ that maps a full-resolution input volume V to a multi-channel heatmap $\mathcal{H}$ encoding (a) per-pixel target likelihood and (b) coarse motion direction classification, such that:
>
> 1. The network preserves full spatial resolution ($H \times W = 1024 \times 1024$) with no spatial downsampling.
> 2. A post-processing step extracts ≤ 15 candidate patches of size $128 \times 128 \times 64$ from $\mathcal{H}$ under nominal operating conditions. As a design margin criterion, the system must achieve $\geq 95\%$ recall at 10 patches and $\geq 99\%$ recall at 20 patches on the validation distribution; meeting the recall@20 target ensures that the operational $\leq$15-patch budget provides robust coverage with margin for threshold variation.
> 3. Peak GPU memory usage during training does not exceed $\sim$45 GB (with gradient checkpointing and mixed-precision activations), leaving the remainder of the 192 GB pipeline budget for Stage 2 inference on candidate patches.
> 4. The architecture generalizes across sensor and scene parameters without hand-tuned components.

The solution to this problem, combined with the existing validated Stage 2 discriminator, yields a complete detection and segmentation pipeline for the full $1024 \times 1024 \times 64$ volume.

3 Solution Overview

This section provides a high-level summary of the Stage 1 design before the detailed theoretical justification (section 4) and architectural specification (section 5). The goal is to orient the reader on *what* the system does and *why* the major design decisions follow from the constraints, so that the subsequent sections can be read as elaboration rather than discovery.

3.1 What Stage 1 Does

Stage 1 is a *learned spatiotemporal region proposal network*. It ingests a single full-resolution input volume $V \in \mathbb{R}^{1024\times1024\times64}$ (spatial $\times$ spatial $\times$ temporal frames) containing calibrated radiance values at float32 precision. It produces a 9-channel spatial heatmap $\mathcal{H} \in \mathbb{R}^{9\times1024\times1024}$: one channel encoding per-pixel target likelihood and eight channels encoding coarse motion direction (uniform 45° bins over $[0, 2\pi)$).

A deterministic post-processing step converts the heatmap into a small set of candidate patches ($\leq$15 per volume) of size $128 \times 128 \times 64$, which are forwarded to the existing validated Stage 2 model for precise segmentation. Stage 1 is optimized for *high recall at low patch count*: the design targets $\geq$95% recall at 10 patches and $\geq$99% recall at 20 patches. Precision is deferred to Stage 2.

3.2 Why Two Stages

The validated Stage 2 discriminator—a 3D convolutional U-Net with a 3D Swin Transformer bottleneck—achieves strong detection and segmentation on $128 \times 128 \times 64$ patches but requires 35–65 GB of GPU memory per patch. It cannot be applied directly to the full $1024 \times 1024 \times 64$ volume, and brute-force tiling into all possible 128×128 patches (225 patches at stride 64) is computationally prohibitive.

Stage 1 exists to solve this mismatch: it processes the full volume at native resolution using a lightweight architecture ($\sim$12 GB peak training memory, $\sim$7,000 parameters), identifies the few spatial regions likely to contain targets, and passes only those regions to the expensive Stage 2 model. The two stages execute sequentially on the same GPU, so the peak pipeline memory is the maximum of the two stages (35–65 GB, dominated by Stage 2), well within the 192 GB budget.

3.3 Design Constraints and Their Consequences

Five constraints from the problem formulation (section 2.4) drive every major architectural decision. Table 1 summarizes the chain from each constraint to the corresponding design choice; the detailed justifications appear in sections 4 and 7.

Table 1: Design constraints and their architectural consequences.

Constraint	**What it eliminates**	**Design consequence**
No spatial downsampling (subpixel energy gradients destroyed by striding/pooling)	U-Nets, FPNs, and all spatial-pyramid detectors	All layers operate at full 1024^2 resolution
Memory budget ($\sim$45 GB for Stage 1 training)	Dense 3D convolutions and spatial self-attention at 1024^2	Per-pixel temporal modeling with deferred spatial mixing
Joint spatiotemporal coupling (motion direction requires correlating neighboring pixels across time)	Purely factored (2+1)D architectures and per-pixel temporal-only models	Spatial mixer re-entangles Mamba features across the PSF neighborhood
Temporal context over all 64 frames ($\mathrm{SNR}_{\mathrm{frame}} < 1$; detection requires $\sqrt{T}$ integration)	Shallow temporal kernels (3D CNNs with k_t=3 need >30 layers) and pairwise optical flow	Mamba SSM spans full temporal window in a single recurrent pass
Generalizability (no hand-tuned parameters)	Classical approaches (registration, median subtraction, CFAR)	Fully learned pipeline; retraining on new synthetic data suffices

The intersection of these constraints leaves a narrow design space: per-pixel temporal sequence modeling (to avoid 1024^2-scale spatial attention or 3D convolutions) followed by a lightweight spatial re-entanglement step (to recover direction information). Within this space, state-space models (Mamba) dominate alternatives on the temporal axis due to $O(N)$ memory per pixel independent of sequence length T, compared to $O(T^2)$ for self-attention.

3.4 Processing Pipeline

Stage 1 processes the input volume through five sequential blocks, illustrated schematically in fig. 1. Tensor shapes are annotated at each transition using the convention (B, C, H, W, T) for spatiotemporal tensors and (B, C, H, W) for spatial-only tensors.

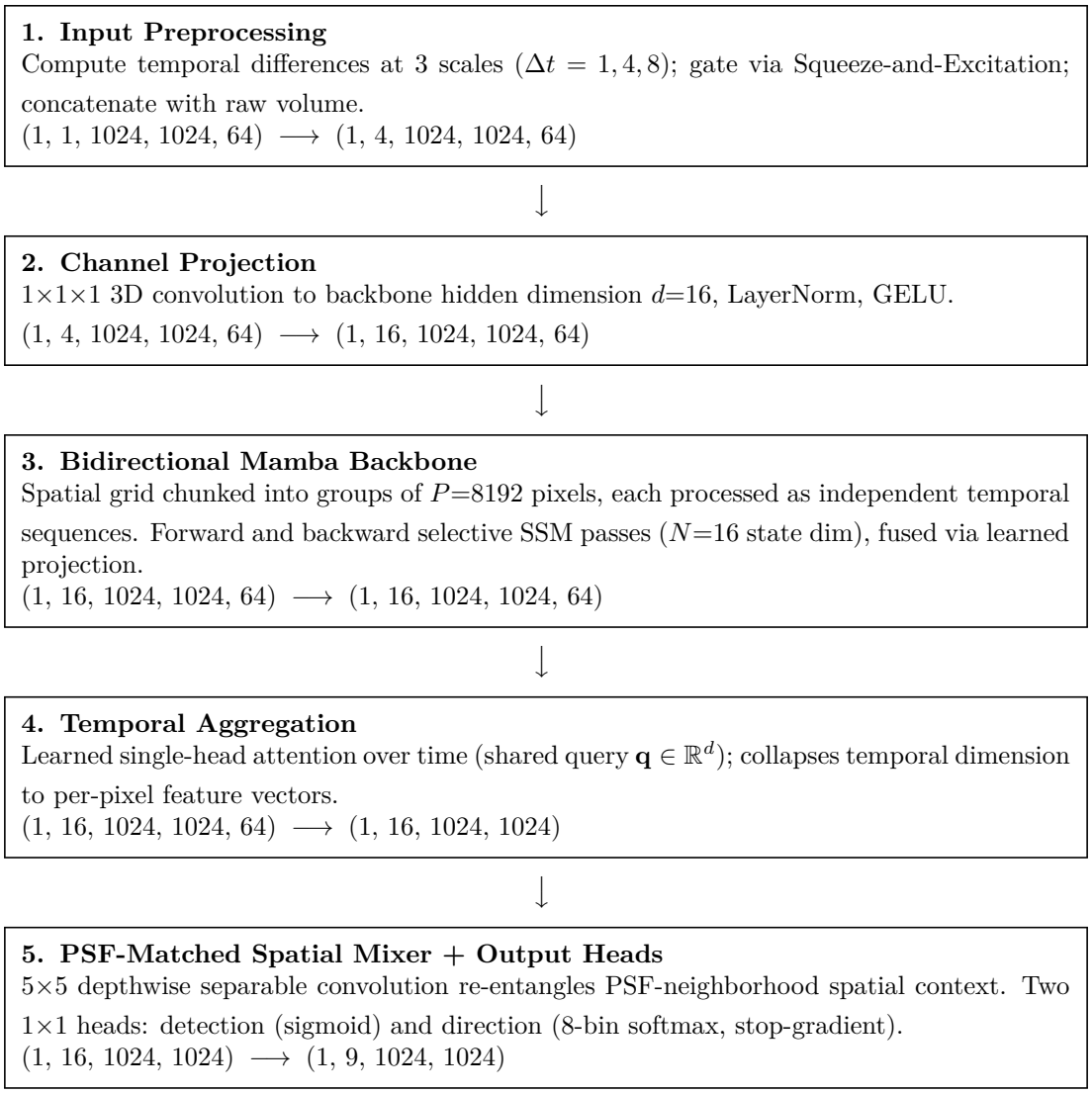


Figure 1: Stage 1 processing pipeline. All spatial dimensions remain 1024×1024 throughout; no downsampling occurs at any point. The temporal dimension (T=64) is preserved through blocks 1–3 and collapsed in block 4.

3.5 Key Design Insights

Three ideas underpin the architecture, each motivated by the physics of the detection problem:

Temporal differences as a motion prior. Multi-scale temporal differences ($\Delta t = 1, 4, 8$) provide an explicit first-order motion signal at no learnable-parameter cost. The SE gating mechanism learns to suppress these channels when sensor jitter dominates (high-jitter regimes), falling back on the Mamba backbone's deeper temporal analysis. This adaptive gating makes the network robust across varying noise conditions without explicit jitter estimation.

Per-pixel SSM with deferred spatial mixing. The Mamba backbone processes each pixel's 64-frame temporal sequence independently, extracting features that encode the temporal dynamics

at that location—accumulation of faint persistent signals, adaptation to local background statistics, and bidirectional evidence fusion across occlusion gaps. Spatial interaction is deferred to a single 5×5 depthwise separable convolution that correlates these temporal features across the PSF footprint. This factorization is *not* the same as (2+1)D convolutions: the spatial mixer operates on rich *temporal feature vectors* rather than raw brightness, so it can detect the spatial correlation patterns (e.g., pixel $\mathbf{p}$ brightening while $\mathbf{p} + \Delta\mathbf{p}$ dims) that encode motion direction.

Motion coherence as a background rejection signal. The 8-bin direction head provides coarse velocity information that the post-processing coherence filter uses to reject cloud false positives. Cloud regions produce spatially uniform direction predictions over large neighborhoods (high coherence); targets produce isolated direction anomalies (low coherence). This two-factor filtering—target likelihood *and* motion incoherence—is what enables the aggressive low detection threshold ($\epsilon_{\text{det}} = 0.1$) needed for high recall without drowning Stage 2 in false-positive patches.

3.6 Post-Processing Summary

The heatmap-to-patch conversion proceeds in three steps (detailed in section 6.2):

1. **Detection thresholding:** Binarize $\mathcal{H}_{\text{det}}$ at a low threshold ($\epsilon_{\text{det}} = 0.1$).
2. **Motion coherence filtering:** Suppress regions where the predicted direction field is spatially uniform over a 64×64 neighborhood (coherence $\Gamma > 0.85$), eliminating cloud-triggered detections.
3. **Connected component extraction:** Label remaining blobs, discard oversized components (>2000 pixels), rank by peak detection score, and extract $128 \times 128 \times 64$ patches centered on each blob's detection-weighted centroid.

The resulting $\leq$15 candidate patches are passed to Stage 2 for segmentation. Stage 1 memory is fully released before Stage 2 begins.

3.7 Training Overview

Stage 1 trains on synthetic data with known ground-truth target positions, velocities, and occlusion states (section 8). The loss combines focal loss on the detection channel (handling extreme class imbalance at $\sim 10^{-4}$ positive fraction) with categorical cross-entropy on the direction channel (applied only at positive pixels, with stop-gradient to decouple from the detection pathway).

Training proceeds in three phases: standalone training on $256 \times 256 \times 64$ tiles (Phase A), scale-up to full $1024 \times 1024 \times 64$ resolution (Phase B), and optional hard-negative mining for Stage 2 (Phase C). The architecture is fully convolutional with no learned position embeddings, so Phase A weights transfer to Phase B without modification. With only ~7,000 parameters, convergence is expected within 30 epochs.

4 Theoretical Foundations

This section presents the theoretical background that motivates and constrains the Stage 1 architecture. We proceed from the sensor physics that govern what information is available in the data, through the computer vision principles that determine how that information can be extracted, to the machine learning foundations that shape the computational architecture.

4.1 Remote Sensing Theory

4.1.1 Point Spread Function and Subpixel Energy Distribution

The optical PSF $h(\mathbf{p})$ is the impulse response of the imaging system: it maps a point source at position $\mathbf{p}_0$ to a spatial distribution $h(\mathbf{p} - \mathbf{p}_0)$ on the focal plane. For a diffraction-limited circular aperture with additional optical aberrations and detector integration effects, the effective system PSF is well approximated by a 2D Gaussian:

$$h(\mathbf{p}) = \frac{1}{2\pi\sigma_{\text{PSF}}^2} \exp\left(-\frac{|\mathbf{p}|^2}{2\sigma_{\text{PSF}}^2}\right), \tag{7}$$

where $\sigma_{\text{PSF}} = w_{\text{PSF}}/2.355$ is the standard deviation corresponding to the FWHM w_{PSF}.

The PSF has two critical consequences for subpixel detection. First, it *distributes* target energy across a neighborhood of radius $\sim 2\sigma_{\text{PSF}}$, reducing the peak pixel intensity relative to the total target flux. For a target centered exactly on a pixel, the fraction of total energy captured by the central pixel is approximately:

$$f_{\text{peak}} = \text{erf}\left(\frac{\Delta}{2\sqrt{2}\,\sigma_{\text{PSF}}}\right)^2, \tag{8}$$

where $\Delta = 1$ pixel is the detector pitch. For $w_{\text{PSF}} = 3$ pixels ($\sigma_{\text{PSF}} \approx 1.27$), $f_{\text{peak}} \approx 0.09$—only about 9% of the target's total radiance falls on the peak pixel. The remaining $\sim$91% is distributed across neighboring pixels, creating the spatially extended but faint footprint that must be recovered by the spatial mixer in the network architecture.

Second, when the target position $\mathbf{p}_k(t)$ shifts by a subpixel displacement $\Delta\mathbf{p}$ between frames, the energy redistribution across the pixel grid changes smoothly. This produces a characteristic *temporal modulation pattern*: each pixel in the PSF footprint experiences a brightness variation that encodes the target's velocity vector $\mathbf{v}_k$. The modulation amplitude at any single pixel is proportional to $|\nabla h| \cdot |\Delta\mathbf{p}|$, which for subpixel shifts ($|\Delta\mathbf{p}| < 1$) is a small fraction of the already-faint PSF peak. This explains why detection requires integrating information across both the spatial PSF neighborhood and the temporal window simultaneously.

4.1.2 Temporal SNR Integration

While a single frame yields $\text{SNR}_{\text{frame}}(t) < 1$ (eq. (6)), temporal integration can boost the effective SNR. For a target at constant velocity $\mathbf{v}_k$ observed over T frames, the ideal matched filter (which correlates the observed volume against a template of the expected trajectory) achieves:

$$\text{SNR}_{\text{matched}} = \text{SNR}_{\text{frame}} \cdot \sqrt{T_{\text{vis}}}, \tag{9}$$

where $T_{\text{vis}} \leq T$ is the number of frames in which the target is visible (not occluded). For $T_{\text{vis}} = 64$ and $\text{SNR}_{\text{frame}} = 0.5$, the matched-filter SNR reaches $\text{SNR}_{\text{matched}} = 4.0$—well above typical detection thresholds.

However, the matched filter requires *a priori* knowledge of the target velocity $\mathbf{v}_k$, which is unknown. A brute-force search over velocity space (the classical track-before-detect approach) is computationally expensive and scales poorly with the number of velocity hypotheses. The Stage 1 network must implicitly learn to perform this velocity-agnostic temporal integration as part of its spatiotemporal feature extraction.

This $\sqrt{T}$ scaling also quantifies the penalty of occlusion: a target occluded for 50% of the window ($T_{\text{vis}} = 32$) suffers a $\sqrt{2} \approx 1.4\times$ SNR reduction relative to the fully visible case. For targets near the detection threshold, this reduction can push the matched-filter SNR below detectability, making occlusion robustness a first-order architectural requirement.

4.1.3 Apparent Motion from Platform Parallax

The sensor platform moves along a trajectory with velocity $\mathbf{v}_{\text{plat}}(t)$ projected onto the ground plane. Objects at different altitudes experience different apparent displacements due to parallax. Under a near-nadir viewing geometry (a simplification; oblique viewing introduces additional angular terms), for a flat terrain at altitude $z = 0$ observed from altitude z_s, the apparent terrain velocity on the focal plane is:

$$\mathbf{v}_{\text{terr}} = \frac{\mathbf{v}_{\text{plat}}}{z_s} \cdot f, \tag{10}$$

where f is the effective focal length in pixel units. Because all terrain features share the same altitude, $\mathbf{v}_{\text{terr}}$ is spatially uniform—the entire terrain translates as a rigid body.

A cloud layer at altitude $z_c < z_s$ has a different parallax-induced velocity:

$$\mathbf{v}^{(l)}_{\text{parallax}} = \frac{\mathbf{v}_{\text{plat}}}{z_s - z_c^{(l)}} \cdot f, \tag{11}$$

in addition to its intrinsic wind-driven velocity $\mathbf{v}^{(l)}_{\text{wind}}$. The total apparent cloud velocity is $\mathbf{v}^{(l)}_{\text{cloud}} = \mathbf{v}^{(l)}_{\text{parallax}} + \mathbf{v}^{(l)}_{\text{wind}}$. Multiple cloud layers at different altitudes therefore move at different velocities, creating a complex multi-velocity background.

The key insight is that all three motion sources—terrain, clouds, and parallax—produce *spatially coherent* motion fields: large contiguous regions of the scene share the same velocity. A target, by contrast, produces a *spatially isolated* velocity anomaly at a single point. This coherence/incoherence dichotomy is the fundamental discriminative signal exploited by Stage 1.

4.1.4 Atmospheric Attenuation and Cloud Interaction

The Beer–Lambert law governs the attenuation of target radiance through an intervening cloud layer. For a cloud with spatially and temporally varying optical depth $\tau^{(l)}(\mathbf{p}, t)$, the transmission factor is:

$$\mathcal{T}^{(l)}(\mathbf{p}, t) = e^{-\tau^{(l)}(\mathbf{p},t)}. \tag{12}$$

For multiple cloud layers, the total transmission is the product:

$$\mathcal{T}_{\text{total}}(\mathbf{p}, t) = \prod_{l=1}^{N_c} \mathcal{T}^{(l)}(\mathbf{p}, t) = \exp\left(- \sum_{l=1}^{N_c} \tau^{(l)}(\mathbf{p}, t) \right). \tag{13}$$

This continuous attenuation model is physically critical and distinct from the binary occlusion model often assumed in tracking literature. As a target moves behind a cloud edge, the optical depth varies smoothly from $\tau \approx 0$ (clear) to the cloud's interior value over a transition zone whose width depends on the cloud's spatial structure. The target signal therefore *fades* rather than vanishing abruptly, producing a characteristic temporal envelope that the network can learn to associate with cloud interaction rather than target absence.

For training purposes, the synthetic data generator must reproduce this continuous attenuation faithfully. Binary on/off occlusion masks will train the network to expect abrupt signal transitions, causing it to miss the gradual-fade case that occurs in practice.

4.2 Computer Vision Theory

4.2.1 Spatiotemporal Feature Extraction

A spatiotemporal feature is a function $\phi : \mathbb{R}^{H \times W \times T} \to \mathbb{R}^d$ that maps a volume (or a local neighborhood within a volume) to a d-dimensional descriptor. For the target detection problem, effective

features must capture the *joint* spatiotemporal structure of the volume—specifically, the correlation between spatial position and temporal evolution at each location.

The simplest spatiotemporal features are separable: a spatial feature $\phi_s(\mathbf{p})$ (e.g., a gradient or texture descriptor) concatenated with a temporal feature $\phi_t(t)$ (e.g., a brightness time series) at each pixel. However, separable features cannot capture the key discriminative signal—motion—because motion is inherently a *coupling* between space and time. A brightness change at pixel $\mathbf{p}$ between frames t and $t+1$ is meaningless without knowing whether the neighboring pixels $\mathbf{p} \pm \Delta\mathbf{p}$ also changed, and in what pattern.

This is the fundamental reason why architectures that factorize spatial and temporal processing (e.g., 2D spatial features followed by 1D temporal aggregation) are insufficient for this problem, as we formalize in section 4.3.1.

4.2.2 Insufficiency of Spatial-Only Features

Spatial features operate on individual frames $V(\cdot, t)$. Classical spatial features (edges, gradients, Gabor filters, HOG descriptors) and learned spatial features (2D CNN activations) detect structure at scales comparable to or larger than the filter kernel. For a subpixel target with $w_{\text{PSF}} = 3$–5 pixels and peak intensity below the noise floor:

1. The target has no spatial morphology at the GSD—it is a smooth, PSF-shaped bump indistinguishable from any other smooth background variation.
2. The target's per-frame intensity is comparable to or below the standard deviation of background fluctuations ($\text{SNR}_{\text{frame}} < 1$), so it cannot be detected by thresholding, local contrast, or anomaly detection on any single frame.
3. Cloud texture, terrain variation, and noise all produce spatial structures at similar scales, creating a massive false-positive space for any spatial-only detector.

No spatial feature, whether hand-crafted or learned, can overcome these limitations because the discriminative information simply does not exist in the spatial domain alone.

4.2.3 Motion Coherence as a Discriminative Signal

The concept of *motion coherence* provides the theoretical basis for target/background discrimination. Define the local motion field at pixel $\mathbf{p}$ as the dominant velocity $\hat{\mathbf{v}}(\mathbf{p})$ estimated from the temporal evolution of $V(\mathbf{p}, t)$ and its spatial neighbors.

For background components, the motion field is *piecewise coherent*: terrain pixels share $\hat{\mathbf{v}} \approx \mathbf{v}_{\text{terr}}$, and cloud pixels in layer l share $\hat{\mathbf{v}} \approx \mathbf{v}_{\text{cloud}}^{(l)}$. These coherent regions are spatially extended (hundreds to thousands of pixels).

A target introduces a *point anomaly* in the motion field: a single location (or PSF-sized neighborhood) where $\hat{\mathbf{v}}$ differs from all surrounding background velocities. The coherence filter in Stage 1's post-processing exploits this directly: regions where the estimated motion direction is spatially uniform over large neighborhoods are classified as background (cloud or terrain), while isolated direction anomalies are candidate targets.

Formally, let $\theta(\mathbf{p})$ be the motion direction at pixel $\mathbf{p}$ predicted by the network. Define the local coherence score:

$$\Gamma(\mathbf{p}) = \frac{1}{|\mathcal{N}(\mathbf{p})|} \sum_{\mathbf{p}' \in \mathcal{N}(\mathbf{p})} \cos\big(\theta(\mathbf{p}) - \theta(\mathbf{p}')\big)\,, \tag{14}$$

where $\mathcal{N}(\mathbf{p})$ is a spatial neighborhood (e.g., 64×64 pixels). Background regions yield $\Gamma(\mathbf{p}) \approx 1$ (uniform direction); target locations yield $\Gamma(\mathbf{p}) \ll 1$ (direction inconsistent with surroundings).

Thresholding on $\Gamma(\mathbf{p})$ combined with the target likelihood channel provides the two-factor filtering that suppresses cloud false positives.

4.2.4 Limitations of Classical Approaches

Several classical computer vision methods have been applied to small-target detection in overhead video. Each has fundamental limitations in the present scenario:

Image registration and differencing. Registering consecutive frames to remove terrain motion, then detecting targets as residuals. This fails when multiple motion layers are present: registration to the terrain velocity leaves cloud residuals that dominate the difference image, producing massive false positives. Multi-layer registration requires estimating $N_c + 1$ velocity fields per frame—itself an unsolved problem when layer boundaries are ambiguous.

Temporal median subtraction. Computing the per-pixel temporal median and subtracting it as a background estimate. This suppresses stationary clutter but fails for moving clouds (the median of a translating texture is spatially blurred, not subtracted) and for slow targets whose trajectory overlaps a small number of pixels (the target contributes to the median at those locations).

Optical flow. Estimating dense optical flow between frames to identify anomalous motion vectors. Classical optical flow (Horn–Schunck, Lucas–Kanade) assumes brightness constancy and spatial smoothness, both violated by: (a) subpixel targets that violate the brightness constancy at sensor noise levels, (b) cloud boundaries where motion is discontinuous, and (c) jitter that introduces global translation. Modern learned optical flow (RAFT, FlowNet) can handle some of these cases but operates on frame pairs, losing the multi-frame temporal integration needed for $\mathrm{SNR}_{\mathrm{frame}} < 1$ detection.

Track-before-detect (TBD). Exhaustively searching over velocity hypotheses to find trajectories that accumulate energy above a threshold. TBD is theoretically optimal (it implements the matched filter of eq. (9)) but computationally prohibitive for the velocity search space at 1024×1024 spatial resolution. For N_v velocity hypotheses, the cost scales as $O(H \times W \times T \times N_v)$, and the number of false alarms grows with $H \times W \times N_v$, requiring careful CFAR thresholding. Furthermore, TBD assumes a parametric motion model (typically constant velocity), which may not hold for maneuvering targets.

All of these methods require explicit parametric assumptions about the scene (number of motion layers, motion model, noise statistics) and hand-tuned thresholds. The learned approach pursued here replaces these assumptions with a data-driven model that adapts to the training distribution, satisfying the generalizability constraint of section 2.4.

4.3 Machine Learning Theory

4.3.1 3D Convolutions and the Factorization Problem

A 3D convolution with kernel $\mathbf{W} \in \mathbb{R}^{C_{\mathrm{out}} \times C_{\mathrm{in}} \times k_s \times k_s \times k_t}$ jointly processes spatial and temporal neighborhoods, learning filters that respond to specific spatiotemporal patterns—including motion at particular velocities and directions. The receptive field after L layers of $k_s \times k_s \times k_t$ kernels spans $L(k_s - 1) + 1$ pixels spatially and $L(k_t - 1) + 1$ frames temporally.

The (2+1)D factorization decomposes this into a spatial 2D convolution followed by a temporal 1D convolution:

$$\mathbf{W}_{3D} \approx \mathbf{W}_{2D} \circ \mathbf{W}_{1D}, \tag{15}$$

reducing the parameter count from $C_{\text{out}} \cdot C_{\text{in}} \cdot k_s^2 \cdot k_t$ to $C_{\text{out}} \cdot C_{\text{in}} \cdot (k_s^2 + k_t)$.

However, this factorization has a critical limitation for subpixel target detection. A 1D temporal convolution at a fixed spatial location computes:

$$y(\mathbf{p}, t) = \sum_{d=-\lfloor k_t/2 \rfloor}^{\lfloor k_t/2 \rfloor} w(d) \cdot x(\mathbf{p}, t+d), \tag{16}$$

which is a weighted sum of brightness values at the *same pixel* across time. This filter cannot distinguish a target moving rightward from one moving leftward through pixel $\mathbf{p}$: both produce a transient brightness increase followed by a decrease. Direction discrimination requires comparing the temporal evolution at $\mathbf{p}$ with that of its spatial neighbors—precisely the joint coupling that factorization eliminates.

For the regime where spatial appearance is identical across all objects (targets, cloud edges, noise artifacts all appear as smooth PSF-scale perturbations), the temporal filter alone cannot determine whether a brightness transient is a moving target, a moving cloud edge, or a noise spike. Only the *spatial pattern* of correlated temporal transients (i.e., a brightness increase at $\mathbf{p}$ followed by one at $\mathbf{p} + \Delta\mathbf{p}$ one frame later) reveals motion direction.

This is why the validated Stage 2 model requires full 3D convolutions, and why Stage 1 must preserve joint spatiotemporal coupling—either through full 3D convolutions (memory-prohibitive at 1024^2) or through an architecture that processes temporal sequences per pixel but re-entangles spatial context afterward (section 5).

4.3.2 State-Space Models for Temporal Sequence Modeling

State-space models (SSMs) provide an alternative to convolutions and attention for processing temporal sequences. The continuous-time linear state-space model is defined by:

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{A}\mathbf{x}(t) + \mathbf{B}u(t), \tag{17}$$

$$y(t) = \mathbf{C}\mathbf{x}(t) + Du(t), \tag{18}$$

where $\mathbf{x}(t) \in \mathbb{R}^N$ is the hidden state, $u(t) \in \mathbb{R}$ is the input, $y(t) \in \mathbb{R}$ is the output, and $(\mathbf{A}, \mathbf{B}, \mathbf{C}, D)$ are the system parameters with $\mathbf{A} \in \mathbb{R}^{N \times N}$, $\mathbf{B} \in \mathbb{R}^{N \times 1}$, $\mathbf{C} \in \mathbb{R}^{1 \times N}$.

For discrete-time processing (as required for frame sequences), the system is discretized via the zero-order hold:

$$\overline{\mathbf{A}} = \exp(\mathbf{A}\Delta), \tag{19}$$

$$\overline{\mathbf{B}} = (\mathbf{A})^{-1}(\overline{\mathbf{A}} - \mathbf{I})\mathbf{B}, \tag{20}$$

where Δ is the discretization step size. The discrete recurrence is:

$$\mathbf{x}_t = \overline{\mathbf{A}}\,\mathbf{x}_{t-1} + \overline{\mathbf{B}}\,u_t, \tag{21}$$

$$y_t = \mathbf{C}\,\mathbf{x}_t + D\,u_t. \tag{22}$$

The parallel scan (SSM scan). The discrete recurrence in eqs. (21) to (22) is inherently sequential: computing $\mathbf{x}_t$ requires $\mathbf{x}_{t-1}$, so a naïve implementation processes the T timesteps one at a time in $O(T)$ serial steps. On massively parallel hardware (GPUs), this sequential bottleneck would negate much of the SSM's memory advantage over attention. The *parallel scan* (also called a *parallel prefix sum*) eliminates this bottleneck by reformulating the recurrence as a sequence of composable operators that can be reduced in a binary tree.

Per-timestep operator. We present the scan for the general case where $\overline{\mathbf{A}}$ and $\overline{\mathbf{B}}$ may vary across timesteps, writing $\overline{\mathbf{A}}_t$ and $\overline{\mathbf{B}}_t$ with explicit time subscripts. (In the linear time-invariant SSM of eqs. (21) to (22), these are constant; the time-varying case arises in the selective Mamba model detailed below.) Each state update $\mathbf{x}_t = \overline{\mathbf{A}}_t\,\mathbf{x}_{t-1} + \overline{\mathbf{B}}_t\,u_t$ is an affine map of the previous state. We represent it as the tuple (or *operator*)

$$o_t = (\overline{\mathbf{A}}_t,\ \ \mathbf{b}_t), \quad \text{where } \mathbf{b}_t \triangleq \overline{\mathbf{B}}_t\,u_t \in \mathbb{R}^N, \tag{23}$$

so that the recurrence reads $\mathbf{x}_t = \overline{\mathbf{A}}_t\,\mathbf{x}_{t-1} + \mathbf{b}_t$. In the selective (Mamba) setting, $\overline{\mathbf{A}}_t$, $\overline{\mathbf{B}}_t$, and u_t are all input-dependent, so each operator o_t varies with the data.

Intuition. Think of each timestep as a machine on a conveyor belt. Frame t's machine takes in the previous hidden state, scales it (via $\overline{\mathbf{A}}_t$—how much of the old memory to retain), and adds new information from the current input (via $\mathbf{b}_t$). The tuple $o_t = (\overline{\mathbf{A}}_t, \mathbf{b}_t)$ is a complete recipe card for that machine: "shrink the old state by this much, then add this much new stuff." Given any input state vector, machine t applies the same deterministic affine transformation—it's fully specified by its recipe card. The parallel scan works by first composing these recipe cards into mega-recipes via a tree reduction—without ever computing an intermediate state vector—and then applying each mega-recipe to the initial state $\mathbf{x}_0$ in parallel to recover all hidden states at once.

Binary composition rule. Given two consecutive operators $o_i = (\overline{\mathbf{A}}_i, \mathbf{b}_i)$ and $o_j = (\overline{\mathbf{A}}_j, \mathbf{b}_j)$ representing timesteps i and $j = i+1$, their composition "apply o_i then o_j" yields a single operator that maps $\mathbf{x}_{i-1}$ directly to $\mathbf{x}_j$ while skipping the intermediate state $\mathbf{x}_i$:

$$o_j \bullet o_i = (\overline{\mathbf{A}}_j\,\overline{\mathbf{A}}_i,\ \ \overline{\mathbf{A}}_j\,\mathbf{b}_i + \mathbf{b}_j). \tag{24}$$

To verify: substituting $\mathbf{x}_i = \overline{\mathbf{A}}_i\,\mathbf{x}_{i-1} + \mathbf{b}_i$ into $\mathbf{x}_j = \overline{\mathbf{A}}_j\,\mathbf{x}_i + \mathbf{b}_j$ gives $\mathbf{x}_j = (\overline{\mathbf{A}}_j\overline{\mathbf{A}}_i)\,\mathbf{x}_{i-1} + (\overline{\mathbf{A}}_j\,\mathbf{b}_i + \mathbf{b}_j)$, which matches the composed tuple.

Intuition. Composition answers the question: "If I skip machine i and go straight from its input to machine j's output, what single recipe card would do the same job?" The composed card $o_j \bullet o_i$ says: shrink the input by both scaling factors multiplied together ($\overline{\mathbf{A}}_j\overline{\mathbf{A}}_i$), and add the combined new information—but the contribution from machine i must itself be scaled by $\overline{\mathbf{A}}_j$, because machine j would have decayed it further. This is exactly what happens if you physically ran the state through both machines in sequence, but now it's captured in a single tuple that you can compute *without* knowing the actual state vector. That decoupling from the state vector is what makes parallelism possible.

Associativity. The composition $\bullet$ is associative: for three consecutive operators o_i, o_j, o_k,

$$(o_k \bullet o_j) \bullet o_i = o_k \bullet (o_j \bullet o_i). \tag{25}$$

This follows from the associativity of matrix multiplication ($\overline{\mathbf{A}}_k(\overline{\mathbf{A}}_j\overline{\mathbf{A}}_i) = (\overline{\mathbf{A}}_k\overline{\mathbf{A}}_j)\overline{\mathbf{A}}_i$) and the linearity of the bias accumulation. Associativity is the essential property: it means we can evaluate

the "running product" of all T operators in *any* grouping order and obtain the same result, which is what enables the parallel tree reduction below.

Intuition. Associativity means the grouping of compositions doesn't matter: composing machines 1–4 then 5–8 gives the same mega-recipe as composing 1–2, 3–4, 5–6, 7–8 and then merging those pairs. This is the property that unlocks tree-structured parallelism. If composition were order-dependent, you'd be forced to compose left-to-right sequentially. Because it's associative, a GPU can split the work into any balanced partition, compose the pieces independently on separate cores, and merge the partial results—arriving at exactly the same answer regardless of how the work was divided.

Tree reduction (the scan algorithm). The goal is to compute every prefix composition $O_t = o_t \bullet o_{t-1} \bullet \cdots \bullet o_1$ for $t = 1, \ldots, T$. Applying O_t to the initial state $\mathbf{x}_0$ yields $\mathbf{x}_t$, so computing all prefix compositions is equivalent to computing all hidden states.

Intuition. The tree reduction is analogous to a tournament bracket (up-sweep) followed by distributing the partial results back down to every participant (down-sweep). In a 64-team tournament, you need only 6 rounds ($\log_2 64$) to crown a champion—not 63 sequential games. Here, each "game" is a composition of two recipe cards into one. The up-sweep is the tournament: after 6 rounds, the root holds the mega-recipe for all 64 timesteps composed together. But we need *every* prefix (the recipe for steps 1–1, 1–2, 1–3, ..., 1–64), not just the final one. The down-sweep distributes the partial results computed during the up-sweep to fill in the missing prefixes—again in $\log_2 T$ rounds, because each intermediate node already holds a partial composition that multiple children need. The total work is $O(T)$ compositions, but spread across only $O(\log T)$ sequential rounds, with massive parallelism within each round.

The parallel prefix scan proceeds in two phases:

Up-sweep (reduce). Arrange the T operators $o_1, \ldots, o_T$ at the leaves of a balanced binary tree. In round $r = 1, 2, \ldots, \lceil \log_2 T \rceil$, compose pairs of partial results from round $r-1$ in parallel using eq. (24). After round r, each node covers a span of 2^r consecutive operators. After $\lceil \log_2 T \rceil$ rounds, the root holds the total composition $o_T \bullet \cdots \bullet o_1$.

Down-sweep (distribute). Propagate partial prefix products back down the tree so that every leaf t receives the full prefix composition O_t. This phase also takes $\lceil \log_2 T \rceil$ rounds, with each round pushing already-computed prefix products to child nodes that need them.

Both phases perform $O(T)$ total composition operations distributed across $2\lceil \log_2 T \rceil$ parallel rounds (each phase contributes $\lceil \log_2 T \rceil$ rounds), giving $O(\log T)$ *parallel depth* (the number of sequential steps on a GPU). For $T = 64$, this means $2 \times \lceil \log_2 64 \rceil = 12$ parallel rounds instead of 64 sequential steps—a $\sim 5\times$ reduction in serial depth.

Concrete example ($T = 8$). To make the algorithm tangible, consider a sequence of 8 timesteps with operators $o_1, \ldots, o_8$. Let $O_{i:j} = o_j \bullet \cdots \bullet o_i$ denote the composition from step i through step j.

Up-sweep:

- Round 1 (4 parallel compositions): compute $O_{1:2}$, $O_{3:4}$, $O_{5:6}$, $O_{7:8}$.
- Round 2 (2 parallel compositions): compose adjacent pairs to get $O_{1:4}$, $O_{5:8}$.
- Round 3 (1 composition): compose to get $O_{1:8}$.

Down-sweep: Propagate the prefix products so that each position t receives $O_{1:t}$:

- $O_{1:1} = o_1$ (trivially available).
- $O_{1:2}$ from round 1.
- $O_{1:3} = O_{3:3} \bullet O_{1:2} = o_3 \bullet O_{1:2}$ (1 composition).

- $O_{1:4}$ from round 2.
- $O_{1:5} = o_5 \bullet O_{1:4}$ (1 composition).
- $O_{1:6} = O_{5:6} \bullet O_{1:4}$ (1 composition).
- $O_{1:7} = o_7 \bullet O_{1:6}$ (1 composition).
- $O_{1:8}$ from round 3.

The down-sweep compositions are again organized into $\lceil \log_2 T \rceil$ parallel rounds, each requiring at most $T/2$ independent compositions. Once all prefix compositions $O_{1:t}$ are available, the hidden states are recovered as $\mathbf{x}_t = \overline{\mathbf{A}}_{1:t}\,\mathbf{x}_0 + \mathbf{b}_{1:t}$, where $(\overline{\mathbf{A}}_{1:t}, \mathbf{b}_{1:t}) = O_{1:t}$.

Intuition for the $T = 8$ example. Notice that in the up-sweep, round 1 does 4 compositions simultaneously (all independent), round 2 does 2, and round 3 does 1—a total of 7 compositions in 3 serial rounds. A sequential approach would need 7 serial steps, each waiting for the previous state. The savings grow with T: for $T = 64$, the full scan (up-sweep plus down-sweep) completes in 12 serial rounds instead of 63 sequential steps. The down-sweep is the "fill in the gaps" phase: the up-sweep gave us $O_{1:2}$, $O_{1:4}$, and $O_{1:8}$, but we're still missing $O_{1:3}$, $O_{1:5}$, $O_{1:6}$, and $O_{1:7}$. Each missing prefix can be built by composing a small leftover piece onto an already-known prefix (e.g., $O_{1:5} = o_5 \bullet O_{1:4}$), and these compositions are again independent and parallelizable.

Why this matters for Stage 1. In this architecture, the Mamba backbone processes $P = 8{,}192$ pixels simultaneously, each as an independent temporal sequence of length $T = 64$. Without the parallel scan, each sequence would require 64 serial state updates—a significant bottleneck when multiplied across $1024^2/P = 128$ sequential chunks. The scan reduces the serial depth to 12 rounds per chunk, making the temporal SSM pass GPU-efficient despite its recurrent definition.

This parallel scan is referred to as the *SSM scan* (or *selective scan* in the Mamba literature, where the input-dependent parameters described below in eq. (26) make the composed tuples data-dependent rather than fixed). During training, the parallel scan is used for both the forward pass and backpropagation (the backward pass through a scan is itself a scan in the reverse direction, applying the chain rule through the same associative structure). During inference, when frames may arrive sequentially, the simpler $O(T)$ recurrence of eqs. (21) to (22) suffices—processing one timestep at a time with $O(N)$ state memory, as quantified below.

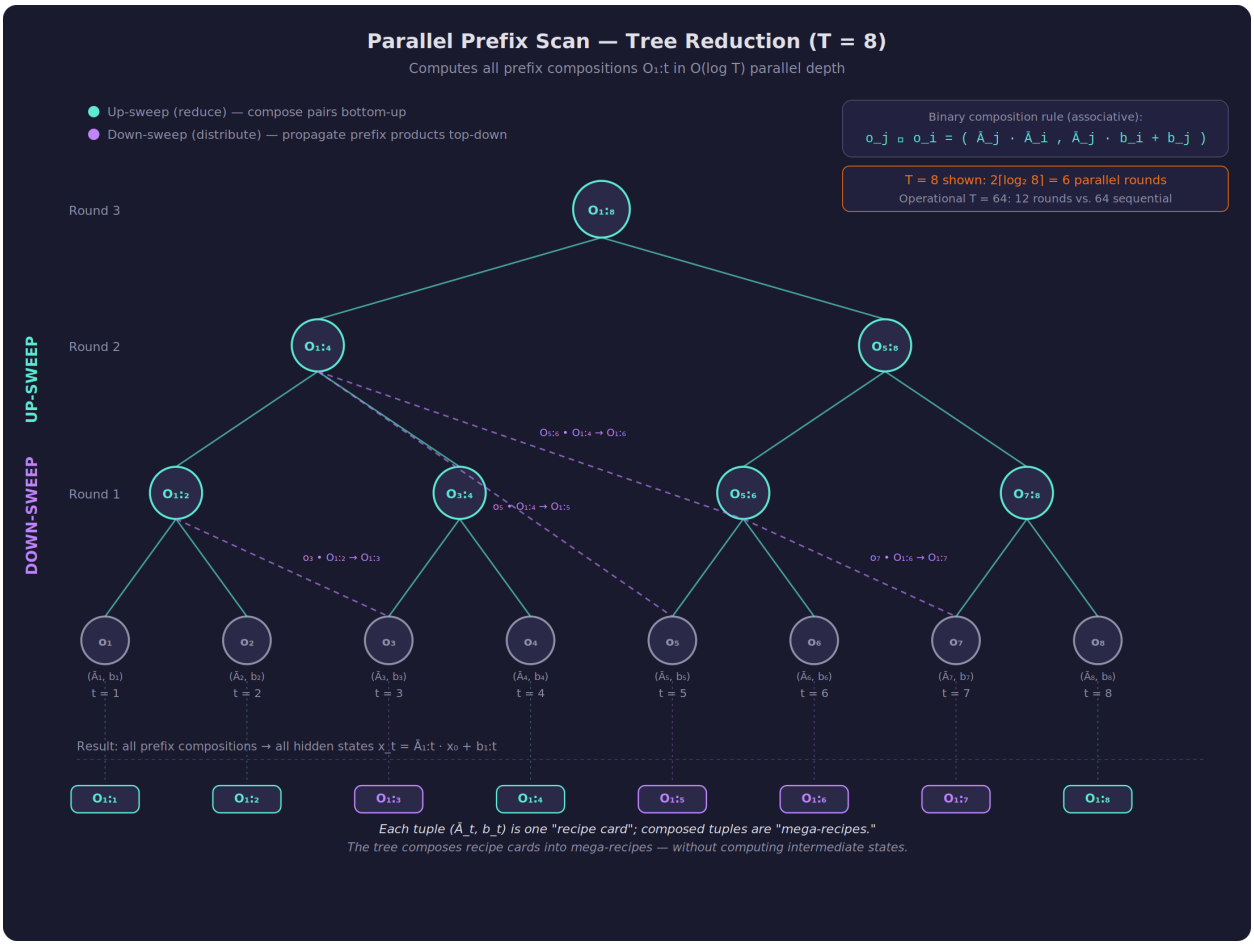


Figure 2: Parallel prefix scan for $T = 8$. Leaf operators $o_t = (\overline{\mathbf{A}}_t, \mathbf{b}_t)$ are composed via the associative binary rule (top right) in a tree reduction. The up-sweep (teal, solid) composes pairs bottom-up in $\lceil \log_2 T \rceil$ rounds; the down-sweep (purple, dashed) propagates prefix products top-down to recover all missing prefixes. The result row shows the complete set of prefix compositions, from which all hidden states are obtained in parallel.

Memory scaling. The critical advantage of SSMs over attention-based temporal models is memory scaling. Self-attention over a sequence of length T requires storing a $T \times T$ attention matrix, yielding $O(T^2)$ memory per spatial location. During inference, the SSM recurrence stores only the current hidden state $\mathbf{x}_t \in \mathbb{R}^N$, yielding $O(N)$ memory per spatial location—independent of sequence length. For $T = 64$ and state dimension $N = 16$, the per-pixel inference cost is 64 bytes (float32), compared to $64^2 \times 4 = 16{,}384$ bytes for the attention matrix. Across 1024^2 spatial locations, SSM inference states occupy $\sim$64 MB vs. $\sim$16 GB for attention.

During training, backpropagation through the recurrence requires storing intermediate states at all T timesteps (absent checkpointing), increasing the per-pixel cost to $N \times T \times 4$ bytes. For $N = 16$ and $T = 64$, this yields $1024^2 \times 16 \times 64 \times 4 \approx 4.3\,\text{GB}$—still a $\sim$4$\times$ reduction relative to attention, and further reducible via gradient checkpointing (recomputing intermediate states during the backward pass at the cost of additional computation).

Selective state-space models (Mamba/S6). The Mamba architecture extends the linear SSM by making the parameters $\mathbf{B}$, $\mathbf{C}$, and Δ *input-dependent*:

$$\mathbf{B}_t = f_B(u_t), \quad \mathbf{C}_t = f_C(u_t), \quad \Delta_t = f_\Delta(u_t), \tag{26}$$

where f_B, f_C, and f_Δ are learned projections. This selectivity allows the model to dynamically control what information enters and leaves the hidden state, enabling content-based reasoning that linear time-invariant SSMs cannot achieve.

For the target detection problem, selectivity is critical: the model must learn to *accumulate* evidence of a faint, persistent brightness perturbation (a target) while *ignoring* transient noise spikes and *adapting* to the local background statistics (cloud brightness, terrain texture). A fixed $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ system would apply the same temporal filter everywhere, unable to distinguish a pixel in a static terrain region from one in a moving cloud field.

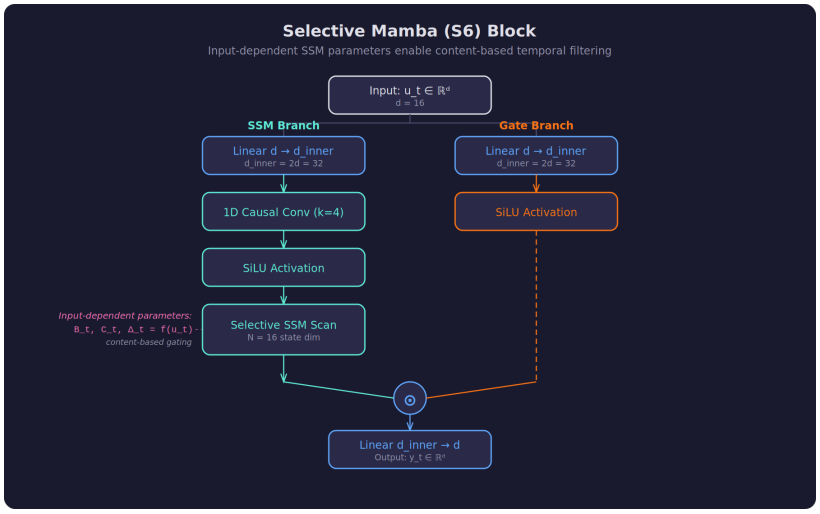


Figure 3: Internal architecture of a single selective Mamba (S6) block. The input is projected into two parallel branches: the SSM branch applies a 1D causal convolution, SiLU activation, and the selective SSM scan (with input-dependent parameters $\mathbf{B}_t$, $\mathbf{C}_t$, Δ_t), while the gate branch applies a SiLU activation. The branches are merged via element-wise multiplication and projected back to the original dimension.

Bidirectional processing. A causal SSM processes the sequence $u_1, \ldots, u_T$ left-to-right, so the output y_t depends only on inputs $u_1, \ldots, u_t$. For offline processing (the full volume is available), a bidirectional SSM runs two passes:

$$y_t^{\text{fwd}} = \text{SSM}_{\text{fwd}}(u_1, \ldots, u_t), \tag{27}$$

$$y_t^{\text{bwd}} = \text{SSM}_{\text{bwd}}(u_T, \ldots, u_t), \tag{28}$$

$$y_t = g(y_t^{\text{fwd}}, y_t^{\text{bwd}}), \tag{29}$$

where g is a learned fusion function. Bidirectional processing is essential for occlusion recovery: when a target is occluded from frame t_1 to t_2, the forward pass can only use pre-occlusion evidence ($t < t_1$) while the backward pass uses post-occlusion evidence ($t > t_2$). Their fusion provides both anchors, enabling detection even when $t_2 - t_1$ exceeds 50% of the temporal window.

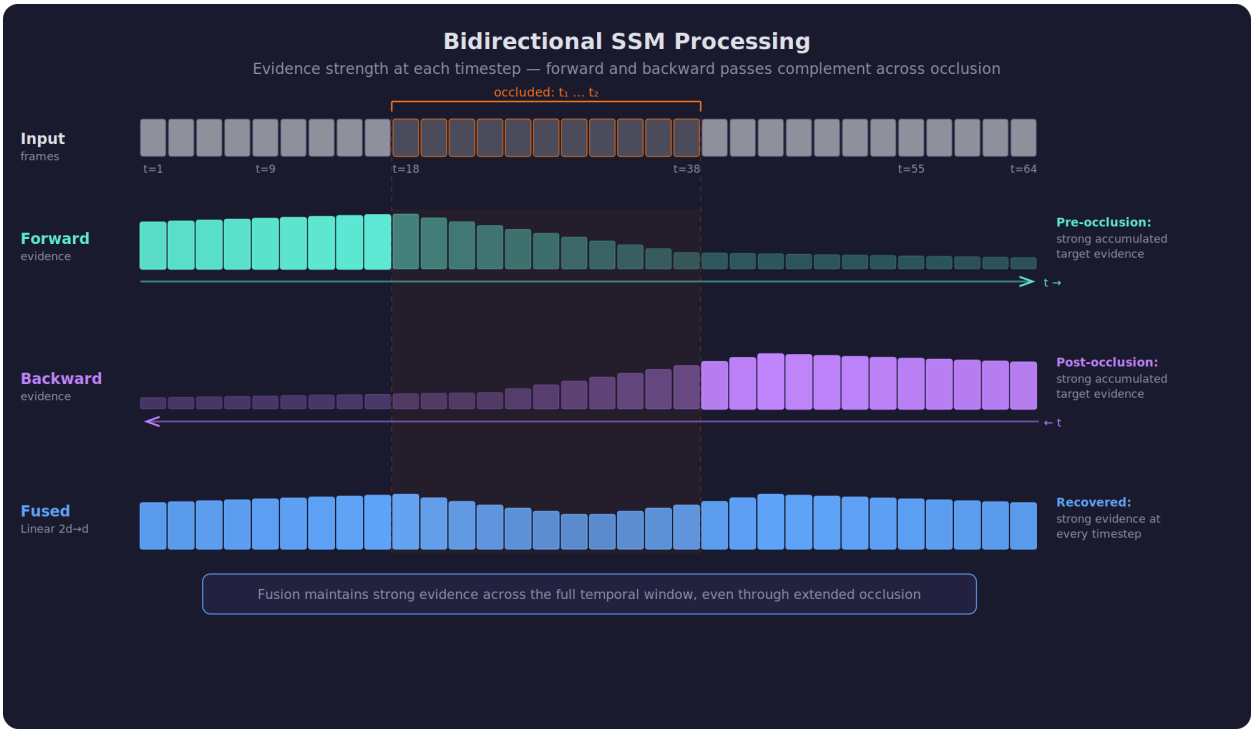


Figure 4: Evidence strength across the temporal window for bidirectional SSM processing. The forward pass (teal) accumulates strong evidence before occlusion but decays through and after the occluded interval. The backward pass (purple) is the mirror: strong after occlusion, weak before. The fused output (blue) combines both passes via a learned linear projection, maintaining strong evidence at every timestep—including through the occluded region where neither pass alone is sufficient.

4.3.3 Channel Attention and Gating Mechanisms

The Squeeze-and-Excitation (SE) block provides a lightweight channel attention mechanism. Given a feature tensor $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$, the SE block computes:

$$\mathbf{z} = \frac{1}{H \cdot W} \sum_{i,j} \mathbf{F}(:, i, j) \in \mathbb{R}^{C}, \tag{30}$$

$$\mathbf{s} = \sigma(\mathbf{W}_2 \, \mathrm{ReLU}(\mathbf{W}_1 \, \mathbf{z})) \in [0, 1]^{C}, \tag{31}$$

$$\hat{\mathbf{F}}(c, :, :) = \mathbf{s}(c) \cdot \mathbf{F}(c, :, :), \tag{32}$$

where $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$ are learned projections with reduction ratio r, and σ is the sigmoid function.

In the Stage 1 architecture, the SE block gates the auxiliary temporal difference channels. When sensor jitter is high, the temporal differences are dominated by jitter-induced global shifts, providing little useful motion information. The SE block learns to suppress these channels (setting $\mathbf{s} \approx 0$ for the TDC channels) and rely on the Mamba backbone's deeper temporal analysis. When jitter is low, the TDC channels provide a strong motion prior, and the SE block passes them through ($\mathbf{s} \approx 1$). This dynamic gating enables graceful adaptation across varying noise conditions without explicit jitter estimation.

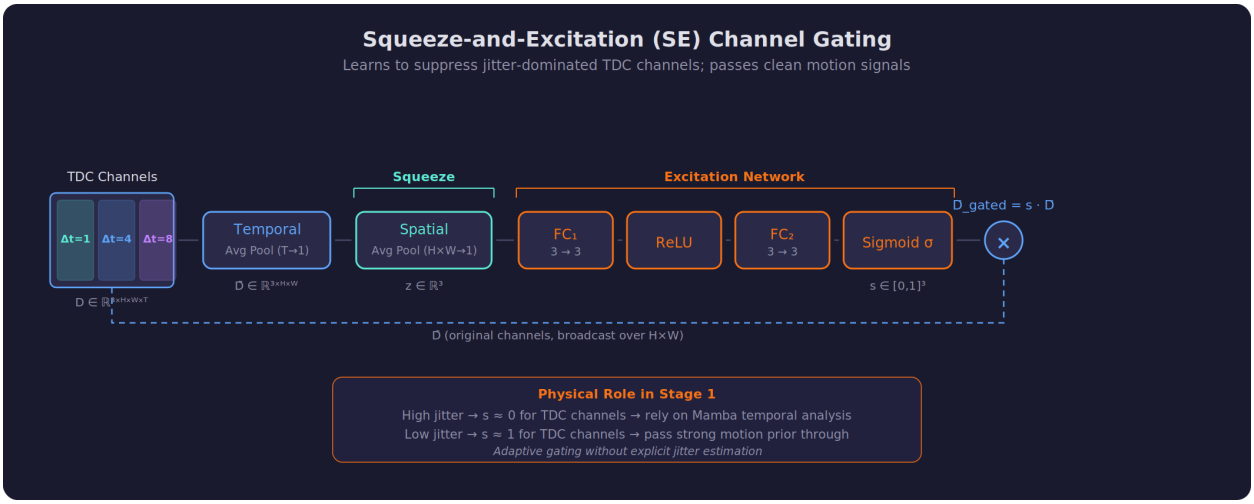


Figure 5: Squeeze-and-Excitation (SE) channel gating applied to the temporal difference channels (TDC). The temporal average pool collapses the temporal dimension; the spatial average pool (squeeze) reduces each channel to a scalar. The excitation network produces per-channel gate values $\mathbf{s} \in [0,1]^3$ that scale the original channels. In high-jitter conditions the gates suppress the TDC channels ($\mathbf{s} \approx 0$); in low-jitter conditions they pass the motion prior through ($\mathbf{s} \approx 1$).

4.3.4 Depthwise Separable Convolutions for Spatial Mixing

A standard 2D convolution with C_{in} input channels, C_{out} output channels, and kernel size $k \times k$ has $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ parameters and performs $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2 \cdot H \cdot W$ multiply-accumulate operations. For the Stage 1 spatial mixer, this is prohibitively expensive at 1024×1024 resolution with per-pixel feature vectors from the Mamba backbone.

Depthwise separable convolution factorizes this into two steps:

1. **Depthwise convolution:** A separate $k \times k$ filter per input channel, with $C_{\text{in}} \cdot k^2$ parameters. This applies spatial filtering independently to each channel, capturing the local spatial structure of individual feature maps.
2. **Pointwise convolution:** A 1×1 convolution with $C_{\text{out}} \cdot C_{\text{in}}$ parameters. This linearly combines the spatially filtered channels, enabling cross-channel interaction.

The total parameter count is $C_{\text{in}} \cdot (k^2 + C_{\text{out}})$, a reduction by a factor of approximately $C_{\text{out}} \cdot k^2/(k^2 + C_{\text{out}})$ relative to the standard convolution. For $k = 5$ and $C_{\text{in}} = C_{\text{out}} = 16$, this is a $\sim 10\times$ reduction.

In the context of Stage 1, the depthwise component serves a specific physical role: it recovers the PSF-distributed target energy by aggregating information across the spatial neighborhood matching the PSF footprint. The kernel size k must therefore satisfy $k \geq \lceil w_{\text{PSF}} \rceil$ to ensure the filter spans the full PSF support. The pointwise component then combines the per-channel spatial features into joint spatiotemporal descriptors that the classification heads can act upon.

4.3.5 Loss Functions for Extreme Class Imbalance

The target detection problem exhibits extreme class imbalance: in a 1024×1024 spatial grid, a target with a 5×5 pixel PSF footprint occupies ~ 25 pixels out of $\sim 10^6$—a positive fraction of 2.5×10^{-5}. With up to 20 targets, the maximum positive fraction is $\sim 5 \times 10^{-4}$, and many scenes contain zero targets.

Standard binary cross-entropy loss is dominated by the overwhelming easy-negative examples (background pixels far from any target), providing negligible gradient signal from the rare positives. Focal loss addresses this by downweighting well-classified examples:

$$\mathcal{L}_{\text{focal}}(p, y) = -\alpha_y \, (1 - p_t)^\gamma \, \log(p_t), \tag{33}$$

where $p_t = p$ if $y = 1$ and $p_t = 1 - p$ if $y = 0$, α_y is a class-balancing weight, and $\gamma \geq 0$ is the focusing parameter. With $\gamma = 2$ (a common choice), an easy negative with $p_t = 0.9$ receives a loss weight of $(1 - 0.9)^2 = 0.01$—a $100\times$ reduction relative to $\gamma = 0$.

For the auxiliary motion direction head, standard categorical cross-entropy over the 8 direction bins is appropriate because the class distribution among positive pixels is approximately uniform across directions. The direction loss is applied only to positive (target-present) pixels, avoiding the imbalance problem entirely.

Critically, the direction classification loss must be *decoupled* from the detection pathway via a stop-gradient operation. Early in training, direction predictions are noisy, and backpropagating their gradients through the shared backbone can destabilize the detection head. The stop-gradient ensures that the backbone features are optimized primarily for detection (the harder task), while the direction head learns to read off coarse velocity information from features that are already useful for detection.

5 Stage 1 Architecture Specification

This section specifies the Stage 1 network in full detail. Tensor shapes are annotated at every stage using the convention (B, C, H, W, T) for spatiotemporal tensors and (B, C, H, W) for spatial-only tensors, where B is the batch dimension (always 1 for full-resolution processing).

5.1 Architecture Overview

The Stage 1 network consists of five sequential blocks:

1. **Input preprocessing:** Compute multi-scale temporal difference channels and gate them via a Squeeze-and-Excitation block.
2. **Channel projection:** Project the augmented input to the backbone's hidden dimension.
3. **Bidirectional Mamba backbone:** Process per-pixel temporal sequences through forward and backward SSM passes, then fuse.
4. **Temporal aggregation:** Reduce the temporal dimension to produce per-pixel feature vectors.
5. **PSF-matched spatial mixer and output heads:** Apply depthwise separable convolution for spatial re-entanglement, then produce the 9-channel heatmap.

The network f_θ maps $V \in \mathbb{R}^{1\times1\times1024\times1024\times64}$ to $\mathcal{H} \in \mathbb{R}^{1\times9\times1024\times1024}$ with no spatial downsampling at any point. The following subsections specify each block.

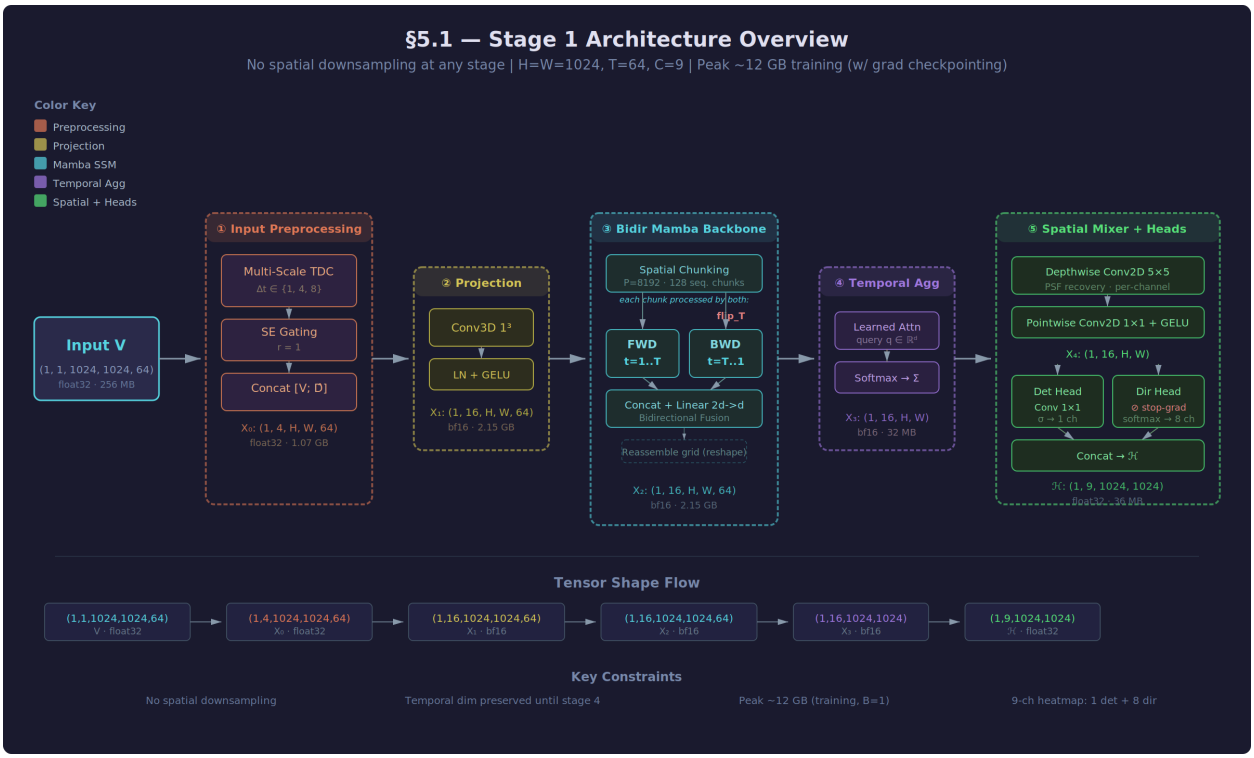


Figure 6: Stage 1 architecture overview. The five sequential blocks are shown with tensor shapes and memory footprints at each transition. The raw input $V \in \mathbb{R}^{1\times1\times1024\times1024\times64}$ is augmented with SE-gated temporal difference channels, projected to the hidden dimension $d = 16$, processed through a chunked bidirectional Mamba backbone, temporally aggregated via learned attention, and spatially mixed before the detection and direction output heads produce the 9-channel heatmap $\mathcal{H}$. No spatial downsampling occurs at any stage; the temporal dimension is preserved through blocks 1–3 and collapsed in block 4. Peak training memory is ∼12 GB with gradient checkpointing.

5.2 Input Preprocessing: Multi-Scale Temporal Differences

Temporal difference computation. Given the input volume $V \in \mathbb{R}^{1\times1\times H\times W\times T}$, compute temporal differences at three scales:

$$D_1(\mathbf{p}, t) = V(\mathbf{p}, t) - V(\mathbf{p}, t-1), \quad t = 2, \ldots, T, \tag{34}$$

$$D_4(\mathbf{p}, t) = V(\mathbf{p}, t) - V(\mathbf{p}, t-4), \quad t = 5, \ldots, T, \tag{35}$$

$$D_8(\mathbf{p}, t) = V(\mathbf{p}, t) - V(\mathbf{p}, t-8), \quad t = 9, \ldots, T. \tag{36}$$

Each difference tensor is zero-padded at the leading temporal edge to restore length T: $\hat{D}_s \in \mathbb{R}^{1\times1\times H\times W\times T}$ for $s \in \{1, 4, 8\}$. The three scales capture motion at different temporal baselines: $\Delta t = 1$ is sensitive to fast motion and jitter; $\Delta t = 4$ captures moderate-speed targets with improved SNR over single-frame differences; $\Delta t = 8$ provides the longest baseline for slow targets (∼0.2–0.5 px/frame), where single-frame differences are dominated by noise.

SE gating. The three difference channels are concatenated into $D \in \mathbb{R}^{1\times3\times H\times W\times T}$. A temporal global average pool reduces this to $\bar{D} \in \mathbb{R}^{1\times3\times H\times W}$, which is then processed by an SE block (section 4.3.3) with reduction ratio $r = 1$ (i.e., the full 3 channels pass through a $3 \to 3 \to 3$

bottleneck). The SE block produces per-channel gate scalars $\mathbf{s} \in [0,1]^3$ (via global average pooling over $H \times W$, followed by the excitation network), which are broadcast across both spatial and temporal dimensions to yield the gated differences $\hat{D}_{\text{gated}} \in \mathbb{R}^{1\times 3\times H\times W\times T}$.

Augmented input. The raw volume and gated differences are concatenated along the channel dimension:

$$X_0 = [V;\ \hat{D}_{\text{gated}}] \in \mathbb{R}^{1\times 4\times H\times W\times T}. \tag{37}$$

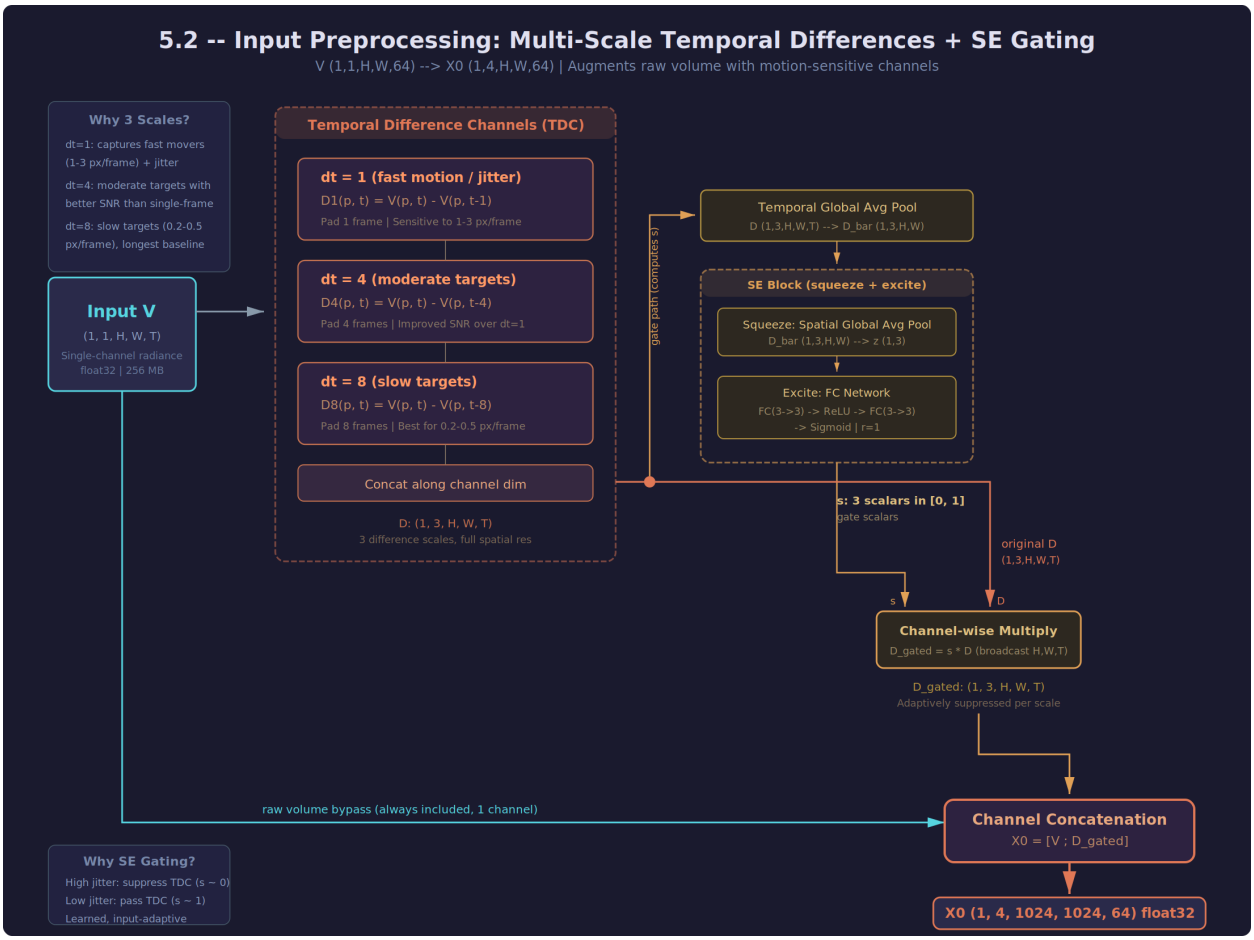


Figure 7: Input preprocessing: multi-scale temporal differences with SE gating. Temporal difference channels (TDC) at three scales ($\Delta t = 1, 4, 8$) are computed from the raw volume V and zero-padded to length T. The three channels are concatenated into $D \in \mathbb{R}^{1\times 3\times H\times W\times T}$, temporally averaged, then passed through a Squeeze-and-Excitation block ($r = 1$) that produces per-channel gate scalars $\mathbf{s} \in [0,1]^3$. The gates adaptively suppress TDC channels under high-jitter conditions ($\mathbf{s} \approx 0$) or pass them through when motion priors are reliable ($\mathbf{s} \approx 1$). The gated differences and the raw volume are concatenated to form $X_0 \in \mathbb{R}^{1\times 4\times H\times W\times T}$.

5.3 Channel Projection

A $1 \times 1 \times 1$ 3D convolution (pointwise in all dimensions) projects the 4-channel augmented input to the backbone hidden dimension d:

$$X_1 = \text{Conv3D}_{1\times 1\times 1}(X_0) \in \mathbb{R}^{1\times d\times H\times W\times T}, \tag{38}$$

with $d = 16$ as the default hidden dimension. This projection is followed by LayerNorm across the channel dimension and a GELU activation. The projection uses float32 weights applied to float32 inputs; subsequent layers operate in bfloat16 for activations (weights remain in float32).

Memory at this stage. X_1 at float32: $16 \times 1024^2 \times 64 \times 4 = 4.29\,\text{GB}$. At bfloat16: 2.15 GB. This is the largest single tensor in the network.

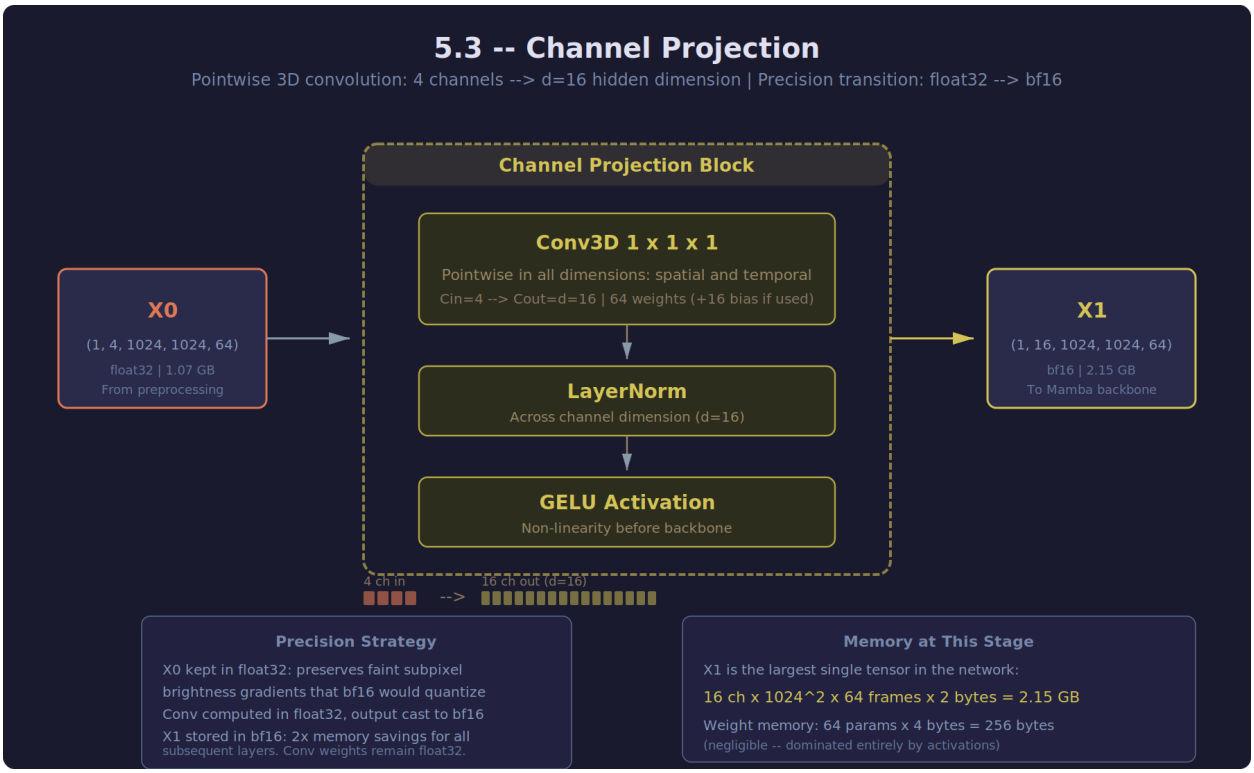


Figure 8: Channel projection block. A pointwise 1×1×1 3D convolution maps the 4-channel augmented input X_0 to the backbone hidden dimension $d = 16$, followed by LayerNorm and GELU activation. The input X_0 is kept in float32 to preserve faint subpixel brightness gradients; the output X_1 is cast to bfloat16, yielding a 2× memory reduction for all subsequent layers. X_1 is the largest single tensor in the network at 2.15 GB. Weight memory is negligible: 64 convolution weights (256 bytes).

5.4 Bidirectional Mamba Backbone

The backbone processes the temporal dimension of X_1 through a bidirectional selective SSM. Because materializing $1024^2 = 1{,}048{,}576$ simultaneous temporal sequences exceeds practical memory limits, the spatial grid is processed in chunks.

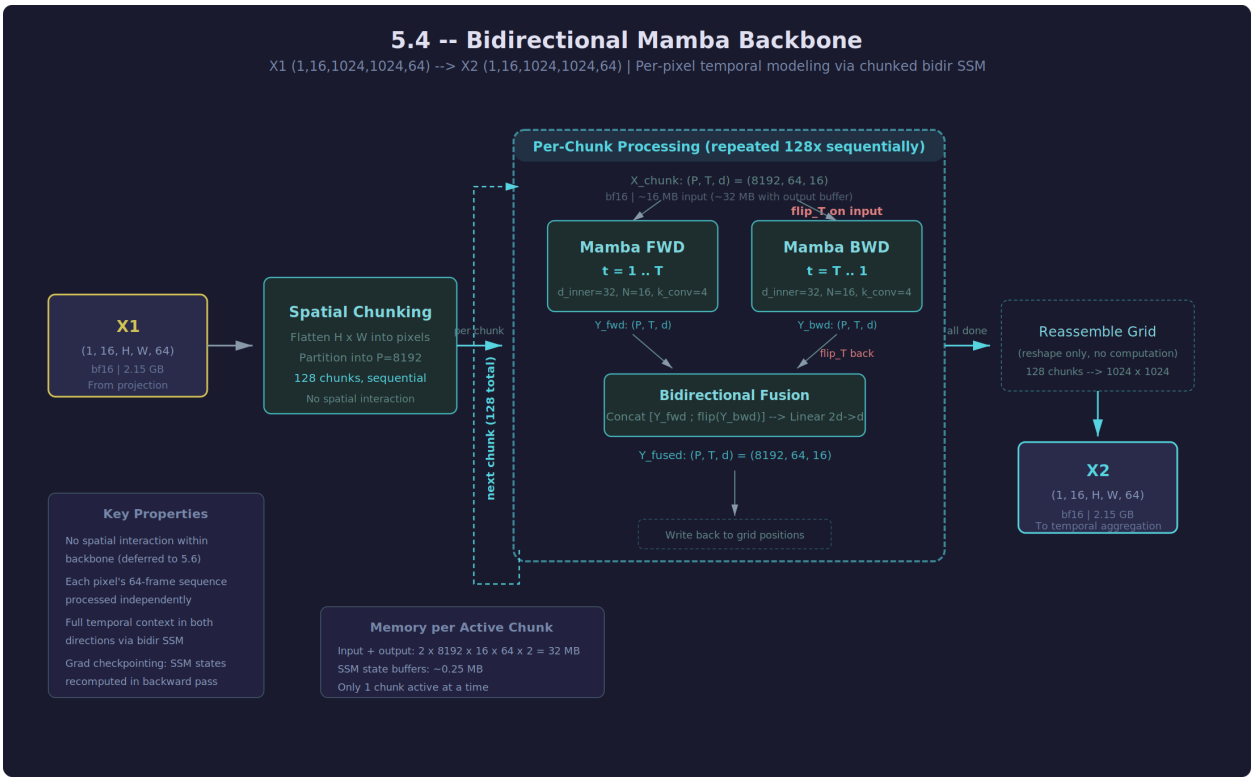


Figure 9: Bidirectional Mamba backbone overview. The projected tensor $X_1 \in \mathbb{R}^{1\times16\times H\times W\times64}$ is spatially chunked into 128 groups of $P = 8192$ pixels. Each chunk is processed sequentially: forward and backward Mamba blocks ($d_{\text{inner}} = 32$, $N = 16$, $k_{\text{conv}} = 4$) extract temporal features in both directions, which are fused via channel concatenation and a learned linear projection ($2d \to d$). Fused outputs are written back to their grid positions and reassembled into $X_2 \in \mathbb{R}^{1\times16\times1024\times1024\times64}$. No spatial interaction occurs within the backbone; only one chunk ($\sim$32 MB) is active at a time.

5.4.1 Spatial Chunking

The spatial dimensions $H \times W$ are partitioned into non-overlapping chunks of P pixels each. For each chunk, the corresponding slice of X_1 is reshaped into a batch of temporal sequences:

$$X_{\text{chunk}} \in \mathbb{R}^{P\times T\times d}, \tag{39}$$

where the P spatial locations are treated as independent batch elements and T is the sequence length for the Mamba forward pass. Chunks are processed sequentially (or in small groups if memory permits), and results are written back to the full spatial grid.

The chunk size P is a tunable parameter governing the memory/compute tradeoff. For $P = 8192$ ($\sim 90 \times 90$ pixels), each chunk requires $8192 \times 16 \times 64 \times 2 \approx 16$ MB of activation memory at bfloat16, plus SSM state overhead. The full grid requires $1024^2/8192 = 128$ chunks processed sequentially.

No spatial information crosses chunk boundaries in this stage—spatial mixing occurs later in the depthwise convolution step. This is acceptable because the Mamba backbone extracts *temporal* features at each spatial location; spatial re-entanglement is deferred to section 5.6.

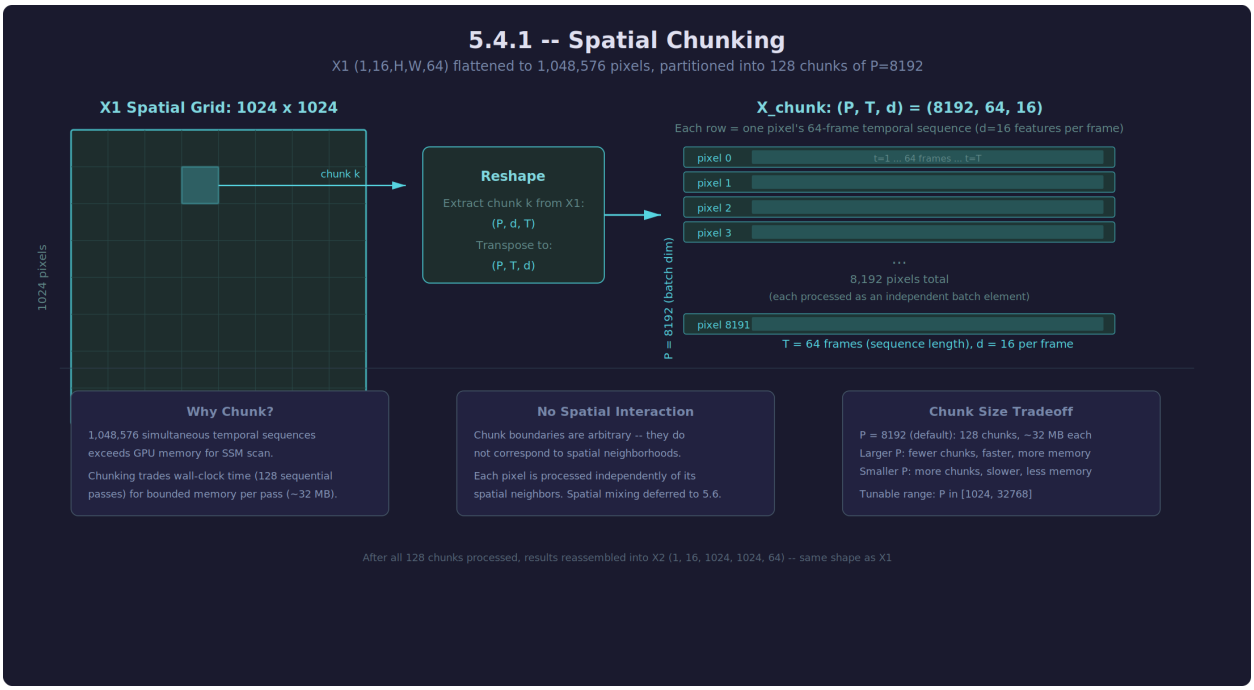


Figure 10: Spatial chunking. The 1024×1024 spatial grid of X_1 is flattened to 1,048,576 pixels and partitioned into 128 non-overlapping chunks of $P = 8192$ pixels each. Each chunk is reshaped to $X_{\text{chunk}} \in \mathbb{R}^{P \times T \times d} = \mathbb{R}^{8192 \times 64 \times 16}$, where the P pixels are treated as independent batch elements and $T = 64$ is the sequence length for the Mamba scan. Chunk boundaries are arbitrary and carry no spatial meaning; no spatial interaction occurs within the backbone. Chunking trades wall-clock time (128 sequential passes) for bounded per-chunk memory ($\sim$32 MB).

5.4.2 Forward and Backward SSM Passes

Each chunk is processed by two independent Mamba blocks:

$$Y^{\text{fwd}} = \text{Mamba}_{\text{fwd}}(X_{\text{chunk}}), \tag{40}$$

$$Y^{\text{bwd}} = \text{Mamba}_{\text{bwd}}(\text{flip}_T(X_{\text{chunk}})), \tag{41}$$

where flip_T reverses the temporal dimension and both outputs are in $\mathbb{R}^{P \times T \times d}$. The backward output is temporally flipped back to the original frame ordering after the SSM pass.

Each Mamba block follows the standard Mamba/S6 architecture:

1. Two parallel linear projections, each $d \to d_{\text{inner}}$ where $d_{\text{inner}} = 2d$ (the "expand" factor), producing a d_{inner}-dimensional SSM branch input and a d_{inner}-dimensional gate branch input.
2. **SSM branch:** 1D causal convolution along the temporal axis with kernel size $k_{\text{conv}} = 4$, followed by SiLU activation and selective SSM scan with state dimension $N = 16$.
3. **Gate branch:** SiLU activation applied to the gate projection output.
4. Element-wise multiplication of the SSM branch and gate branch outputs, followed by output projection $d_{\text{inner}} \to d$.

Mamba hyperparameters. The default configuration uses $d = 16$, $d_{\text{inner}} = 32$, $N = 16$, and $k_{\text{conv}} = 4$. This is deliberately narrow: Stage 1 is a proposal network optimized for recall, not a full discriminator. The narrow width keeps memory tractable at 1024^2 spatial scale.

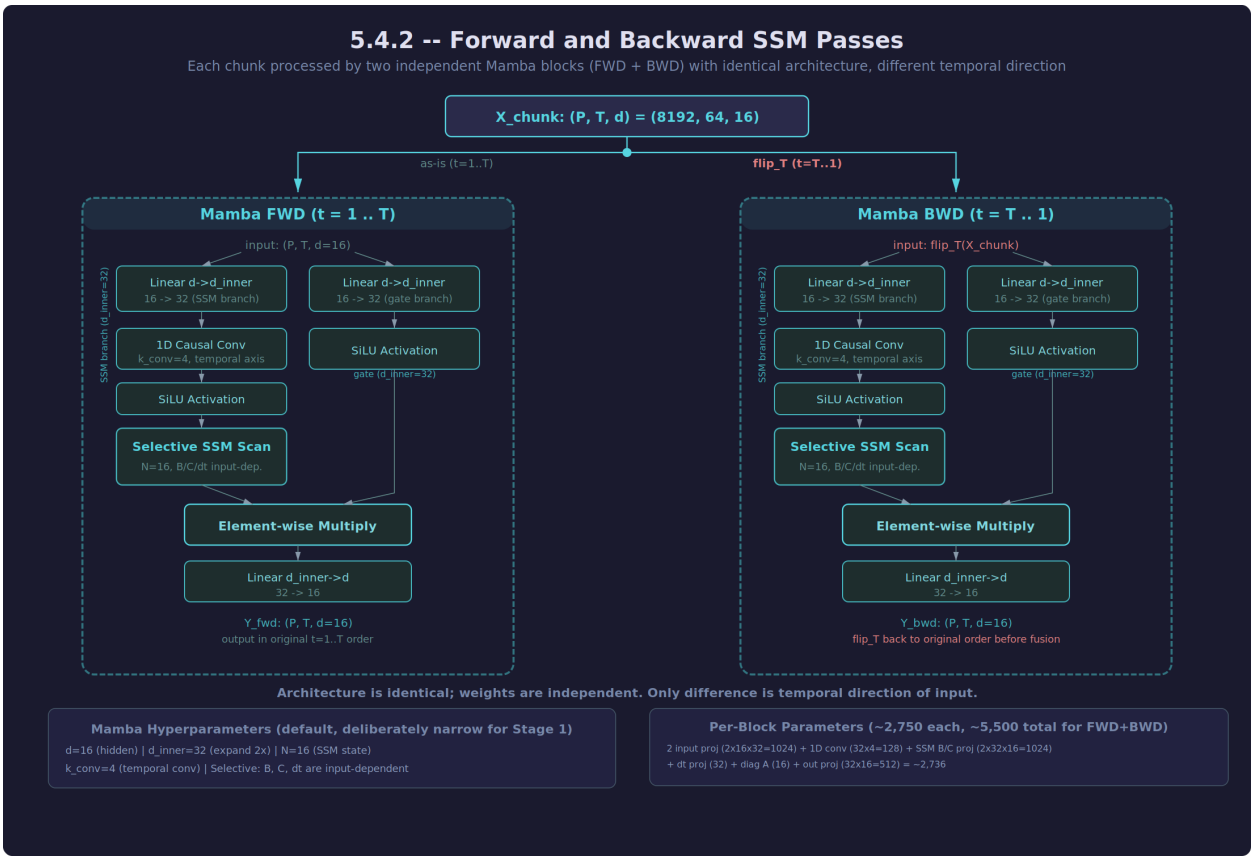


Figure 11: Forward and backward SSM passes. Each chunk $X_{\text{chunk}} \in \mathbb{R}^{P\times T\times d}$ is processed by two independent Mamba blocks with identical architecture but separate weights. The forward block receives the chunk in original temporal order ($t = 1 \dots T$); the backward block receives a temporally flipped copy ($t = T \dots 1$). Each block follows the standard Mamba/S6 design: dual linear projections to $d_{\text{inner}} = 32$, a 1D causal convolution ($k_{\text{conv}} = 4$) and selective SSM scan ($N = 16$) on the SSM branch, SiLU gating on the gate branch, element-wise multiplication, and output projection back to $d = 16$. Both blocks produce outputs in $\mathbb{R}^{P\times T\times d}$; the backward output is flipped back to original order before fusion. Total parameters: ∼2,750 per block, ∼5,500 for the pair.

5.4.3 Bidirectional Fusion

The forward and backward outputs are concatenated along the channel dimension and fused via a pointwise projection:

$$Y = \text{Linear}_{2d\to d}([Y^{\text{fwd}};\ \text{flip}_T(Y^{\text{bwd}})]) \in \mathbb{R}^{P\times T\times d}, \tag{42}$$

where the linear projection from $2d \to d$ is applied independently at each timestep.

After all spatial chunks are processed, the fused output is reassembled into:

$$X_2 \in \mathbb{R}^{1\times d\times H\times W\times T}. \tag{43}$$

This tensor encodes, at each spatial location, a d-dimensional feature vector at each of the T timesteps, informed by the full temporal context in both directions.

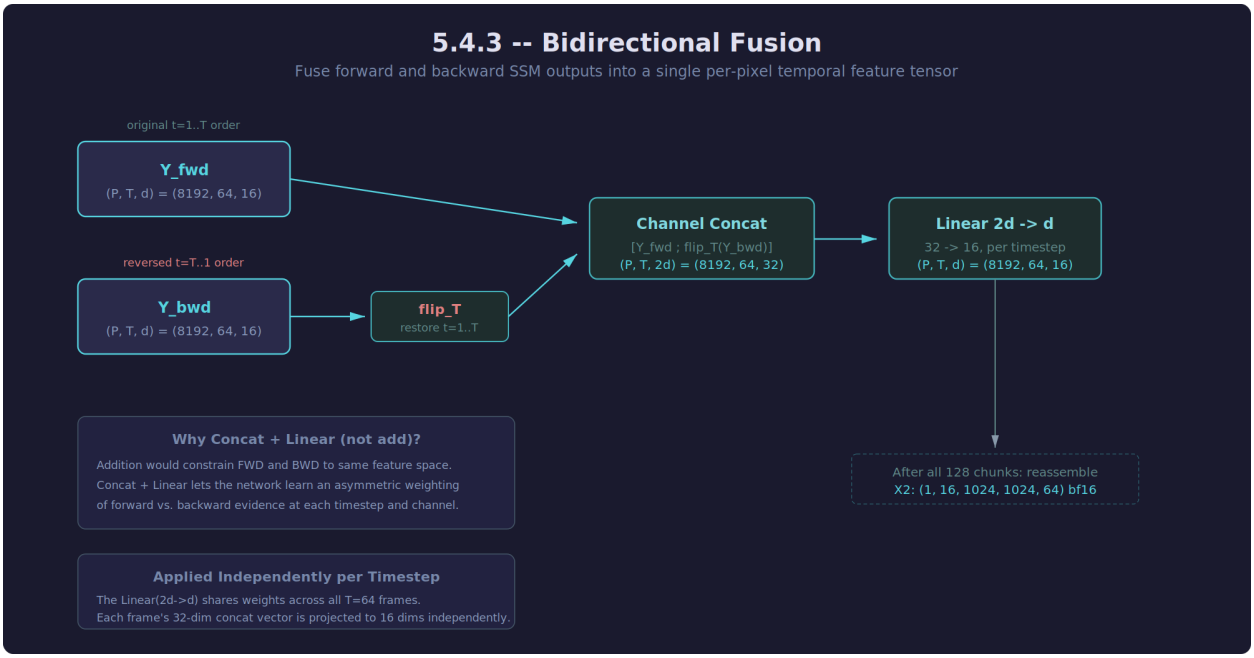


Figure 12: Bidirectional fusion. The forward output Y^{fwd} (in original temporal order) and the temporally re-flipped backward output $\text{flip}_T(Y^{\text{bwd}})$ are concatenated along the channel dimension to produce a $(P, T, 2d)$ tensor, then projected back to d dimensions by a linear layer applied independently per timestep. Concatenation followed by learned projection—rather than simple addition—allows the network to learn asymmetric weighting of forward and backward evidence at each channel and timestep. After all 128 spatial chunks are processed, results are reassembled into $X_2 \in \mathbb{R}^{1\times16\times1024\times1024\times64}$.

5.5 Temporal Aggregation

The heatmap output is spatial-only ($H \times W$), so the temporal dimension must be reduced. We use a *learned temporal aggregation* rather than simple averaging (which discards temporal structure) or final-state extraction (which discards intermediate evidence):

$$X_3(\mathbf{p}) = \sum_{t=1}^{T} \alpha_t(\mathbf{p}) \cdot X_2(\mathbf{p}, t), \quad \text{where} \quad \alpha_t(\mathbf{p}) = \frac{\exp(\mathbf{q}^\top X_2(\mathbf{p}, t))}{\sum_{t'=1}^{T} \exp(\mathbf{q}^\top X_2(\mathbf{p}, t'))}, \tag{44}$$

where $\mathbf{q} \in \mathbb{R}^d$ is a learned query vector shared across all spatial locations, and $\alpha_t(\mathbf{p})$ are softmax attention weights over time.

This is a single-head temporal attention with $O(d)$ parameters, producing:

$$X_3 \in \mathbb{R}^{1\times d\times H\times W}. \tag{45}$$

The attention weights $\alpha_t(\mathbf{p})$ allow the network to upweight frames where the target is visible and downweight occluded frames, directly addressing the occlusion robustness requirement. Unlike mean pooling, this is *spatially varying*: at a pixel where a target appears only in frames 10–30, the attention concentrates on those frames; at a background pixel, the attention distributes more uniformly.

Memory. X_3 at bfloat16: $16 \times 1024^2 \times 2 = 32\,\text{MB}$—negligible after temporal collapse.

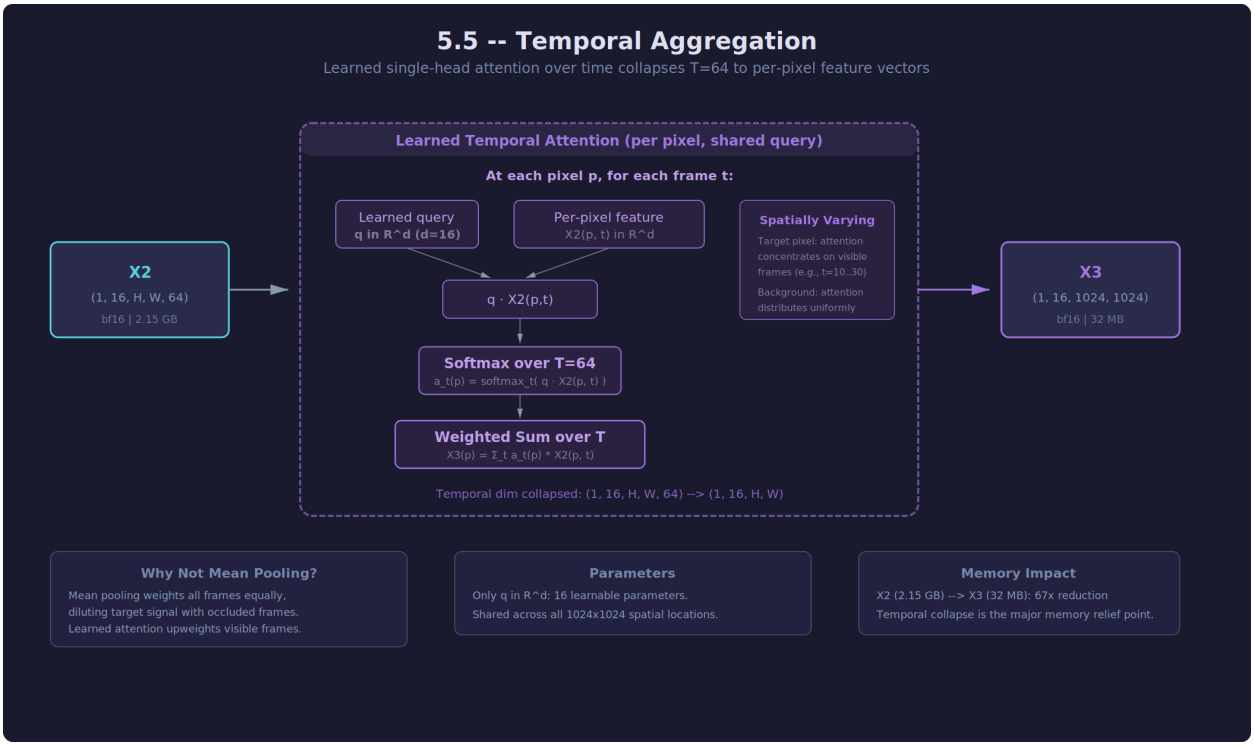


Figure 13: Learned temporal aggregation. A shared query vector $\mathbf{q} \in \mathbb{R}^d$ is dotted with each per-pixel temporal feature $X_2(\mathbf{p}, t)$, producing softmax attention weights $\alpha_t(\mathbf{p})$ over the T=64 frames. The weighted sum collapses the temporal dimension from $X_2 \in \mathbb{R}^{1\times16\times H\times W\times64}$ to $X_3 \in \mathbb{R}^{1\times16\times H\times W}$—a 67× memory reduction. The attention is spatially varying: target pixels concentrate weight on visible frames while background pixels distribute weight more uniformly. Only d=16 learnable parameters are introduced.

5.6 PSF-Matched Spatial Mixer

At this point, each pixel has a d-dimensional temporal feature vector, but pixels are spatially independent (the Mamba backbone processed each pixel's temporal sequence without spatial interaction). The spatial mixer re-entangles local spatial neighborhoods to recover PSF-distributed target energy and enable motion direction discrimination (section 4.1.1, section 4.3.1).

Depthwise convolution (PSF recovery). A depthwise 2D convolution with kernel size $k_{\text{PSF}} \times k_{\text{PSF}}$ is applied independently per channel:

$$X_4^{\text{dw}} = \text{DepthwiseConv2D}_{k_{\text{PSF}}\times k_{\text{PSF}}}(X_3) \in \mathbb{R}^{1\times d\times H\times W}, \tag{46}$$

with $k_{\text{PSF}} = 5$ as the default (matching a PSF FWHM of $w_{\text{PSF}} \leq 5$ pixels; adjust to $k_{\text{PSF}} = 3$ if the sensor PSF is narrower or $k_{\text{PSF}} = 7$ if wider). Padding is set to $\lfloor k_{\text{PSF}}/2 \rfloor$ to preserve spatial dimensions.

Pointwise convolution (channel mixing). A 1×1 convolution combines the spatially filtered channels:

$$X_4 = \text{GELU}\Big(\text{Conv2D}_{1\times1}(X_4^{\text{dw}})\Big) \in \mathbb{R}^{1\times d\times H\times W}, \tag{47}$$

mapping $d \to d$ channels. The combination of depthwise and pointwise convolutions forms a depthwise separable block (section 4.3.4) with $d \cdot (k_{\text{PSF}}^2 + d) = 16 \cdot (25 + 16) = 656$ parameters.

Physical interpretation. The depthwise filters learn to aggregate the temporal features of neighboring pixels—effectively correlating the Mamba-extracted temporal signatures across the PSF footprint. A target produces correlated temporal transients across the PSF neighborhood (brightness shifting smoothly from pixel to pixel as the target moves); background noise produces uncorrelated temporal features at neighboring pixels. The depthwise convolution amplifies the correlated signal and suppresses the uncorrelated noise. The pointwise convolution then projects these spatially aggregated features into a representation suitable for the classification heads.

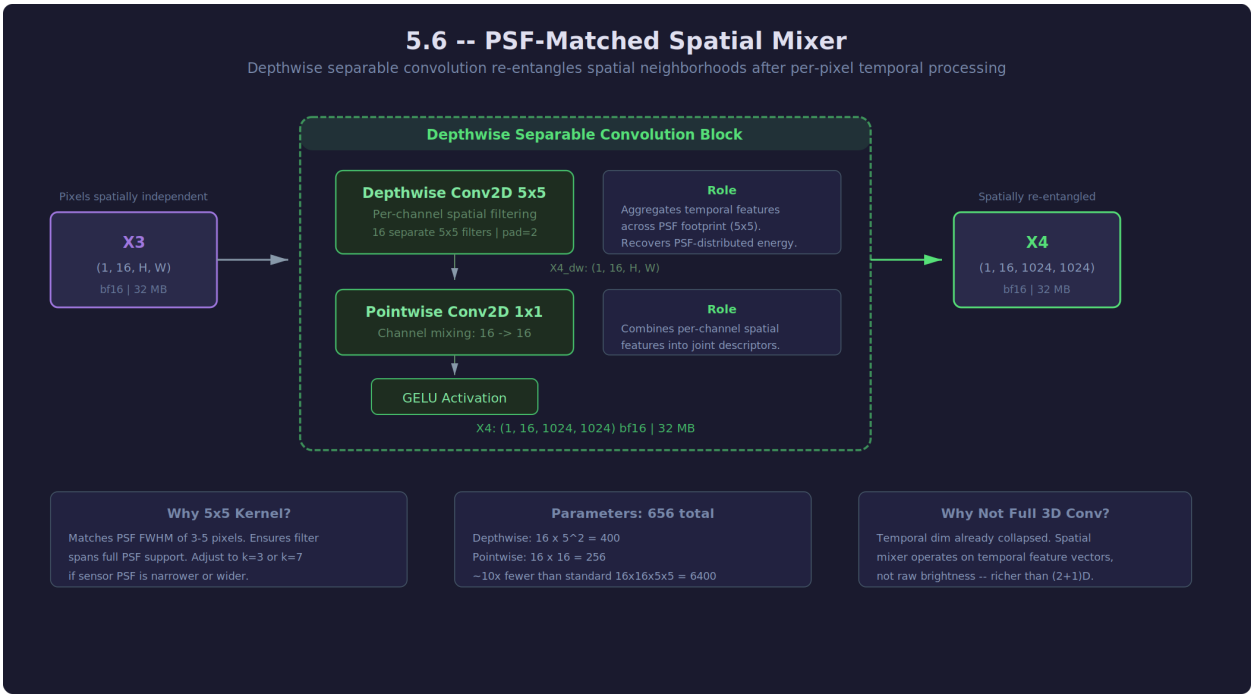


Figure 14: PSF-matched spatial mixer. A depthwise separable convolution block re-entangles spatial neighborhoods after per-pixel temporal processing. The 5×5 depthwise convolution applies 16 independent spatial filters (one per channel) matching the PSF footprint, followed by a 1×1 pointwise convolution for channel mixing and a GELU activation. Input $X_3 \in \mathbb{R}^{1\times16\times H\times W}$ maps to X_4 with the same shape, using only 656 parameters—approximately 10× fewer than a standard 16×16×5×5 convolution.

5.7 Output Heads

Two parallel 1×1 convolutional heads produce the multi-channel heatmap from X_4:

Detection head.

$$\mathcal{H}_{\text{det}} = \sigma\Big(\text{Conv2D}_{1\times1}^{(\text{det})}(X_4)\Big) \in \mathbb{R}^{1\times1\times H\times W}, \tag{48}$$

where σ is the sigmoid function. Each pixel value represents the estimated probability that a target is present at (or within the PSF neighborhood of) that location during any frame in the temporal window.

Direction head.

$$\mathcal{H}_{\text{dir}} = \text{Softmax}_C\Big(\text{Conv2D}_{1\times1}^{(\text{dir})}(\text{sg}(X_4))\Big) \in \mathbb{R}^{1\times8\times H\times W}, \tag{49}$$

where $\text{sg}(\cdot)$ denotes the stop-gradient operator (blocks gradient flow from the direction loss into the backbone) and Softmax_C is applied across the 8 direction bins. The 8 bins partition $[0, 2\pi)$ into uniform 45° sectors. The direction prediction is meaningful only at pixels where $\mathcal{H}_{\text{det}} > \epsilon$ for some threshold ϵ; at background pixels, the direction output is ignored.

Combined output. The full heatmap is the channel-wise concatenation:

$$\mathcal{H} = [\mathcal{H}_{\text{det}};\ \mathcal{H}_{\text{dir}}] \in \mathbb{R}^{1\times9\times H\times W}. \tag{50}$$

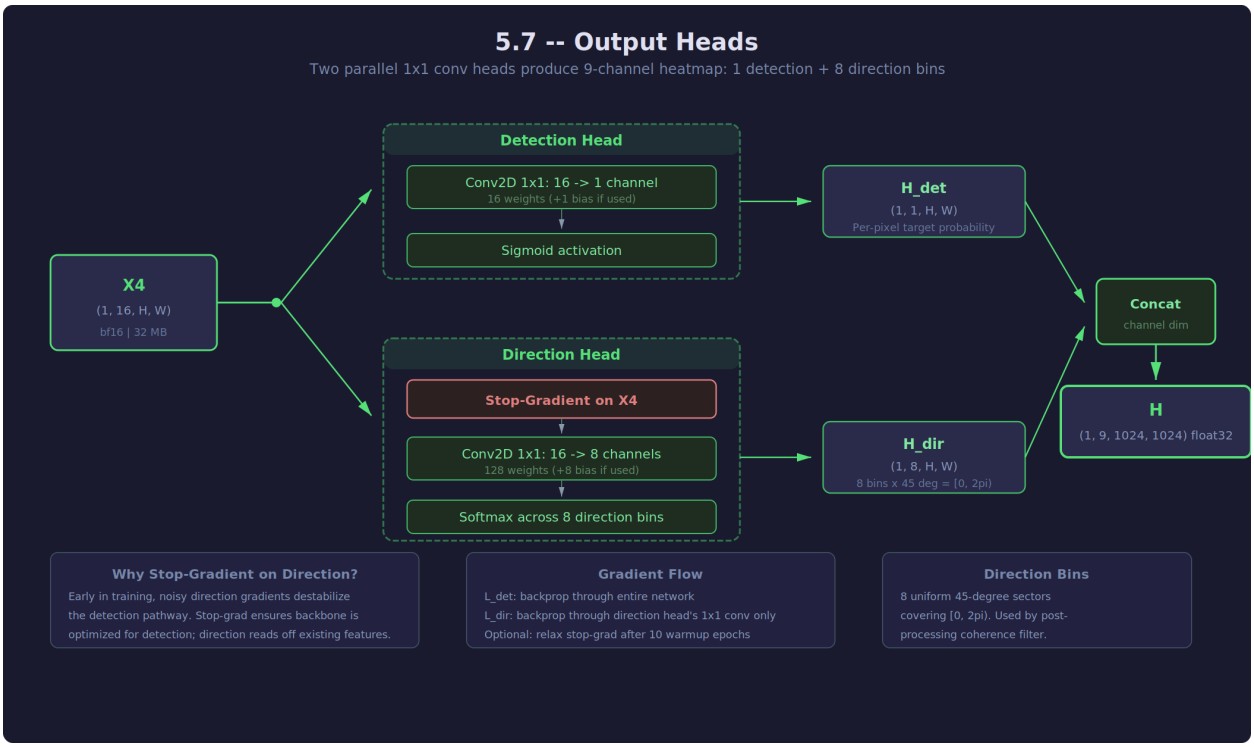


Figure 15: Output head architecture. Two parallel 1×1 convolutional heads operate on the spatially mixed features X_4: the detection head applies sigmoid activation to produce per-pixel target probability $\mathcal{H}_{\text{det}} \in \mathbb{R}^{1\times1\times H\times W}$, while the direction head applies a stop-gradient on X_4 followed by softmax across 8 direction bins to produce $\mathcal{H}_{\text{dir}} \in \mathbb{R}^{1\times8\times H\times W}$. The outputs are concatenated into the 9-channel heatmap $\mathcal{H}$. The stop-gradient ensures that direction loss gradients do not propagate into the backbone during training.

5.8 Memory Analysis

We derive the peak GPU memory for Stage 1 training with batch size 1, bfloat16 activations, float32 weights, and gradient checkpointing on the Mamba backbone.

5.8.1 Activation Memory

Input and TDC. The raw input V (float32): $1024^2 \times 64 \times 4 = 256\,\text{MB}$. The TDC channels and gated output (float32, to preserve faint gradients): $3 \times 1024^2 \times 64 \times 4 \approx 0.81\,\text{GB}$, plus SE intermediates. Augmented input X_0: $4 \times 1024^2 \times 64 \times 4 = 1.07\,\text{GB}$. Once X_0 is constructed, V and the raw TDC tensors can be freed; X_0 is retained for the projection layer's backward pass.

Projected tensor X_1. At bfloat16: $16 \times 1024^2 \times 64 \times 2 = 2.15\,\text{GB}$. This is the dominant stored activation.

Mamba backbone (with gradient checkpointing). With checkpointing, intermediate SSM states are not stored during the forward pass; they are recomputed during backpropagation. The stored memory per chunk is limited to the chunk input and output: $2 \times P \times d \times T \times 2$ bytes (bfloat16). For $P = 8192$: $2 \times 8192 \times 16 \times 64 \times 2 = 32\,\text{MB}$ per chunk. Only one chunk is active at a time, so backbone memory is $\sim$32 MB plus the SSM scan buffers ($\sim P \times N \times 2 \approx 0.25\,\text{MB}$).

The full output X_2 must be stored: $16 \times 1024^2 \times 64 \times 2 = 2.15\,\text{GB}$.

Post-temporal-aggregation. X_3 and X_4 are spatial-only at bfloat16: $16 \times 1024^2 \times 2 = 32\,\text{MB}$ each. Output $\mathcal{H}$: $9 \times 1024^2 \times 4 = 36\,\text{MB}$ (float32 for loss computation).

5.8.2 Weight and Optimizer Memory

Total parameters are dominated by the two Mamba blocks. Each Mamba block with $d = 16$, $d_{\text{inner}} = 32$, $N = 16$: two input projections ($2 \times d \times d_{\text{inner}} = 1024$), 1D convolution ($d_{\text{inner}} \times k_{\text{conv}} = 128$), SSM parameters (**B** and **C** projections $2 \times d_{\text{inner}} \times N = 1024$, Δ projection $d_{\text{inner}} \times 1 = 32$, diagonal **A**: $N = 16$), and output projection ($d_{\text{inner}} \times d = 512$), totaling $\sim$2,750 parameters (exact count varies with implementation details). Two blocks (forward + backward): $\sim$5,500 parameters. All other layers (fusion, spatial mixer, heads): $\sim$1,500 parameters. Total: $\sim$7,000 parameters—negligible. With Adam optimizer (2 state tensors per parameter) at float32: $7{,}000 \times 12 \approx 0.1\,\text{MB}$.

5.8.3 Peak Memory Estimate

Table 2: Stage 1 peak memory breakdown (training, batch size 1). Peak occurs during backpropagation through the channel projection, when X_0, X_1, and X_2 must coexist with gradient tensors. Input V and TDC intermediates are freed after X_0 is constructed.

Component	**Precision**	**Size**
Augmented input X_0	float32	1.07 GB
Projected X_1	bfloat16	2.15 GB
Mamba output X_2	bfloat16	2.15 GB
Active chunk buffers	bfloat16	0.03 GB
X_3, X_4, $\mathcal{H}$	mixed	0.13 GB
Gradient tensors (for X_1, X_2)	bfloat16	4.30 GB
Framework overhead ($\sim$20%)	—	2.0 GB
Estimated peak		**$\sim$12 GB**

This estimate assumes aggressive gradient checkpointing on the Mamba backbone (recomputing SSM states during backprop rather than storing them) and bfloat16 activations throughout the backbone. The float32 input and TDC channels are necessary to preserve the faint subpixel brightness gradients that would be quantized away at lower precision.

The $\sim$12 GB peak is substantially below the 45 GB budget specified in section 2.5, providing headroom for:

- Increasing the hidden dimension d (e.g., $d = 32$ roughly doubles activation memory to $\sim$20 GB).
- Adding Mamba layers (stacking 2–3 bidirectional blocks instead of 1).
- Reducing chunk aggressiveness (larger P for fewer sequential passes at the cost of higher per-chunk memory).

5.9 Architectural Degrees of Freedom

Several parameters are tunable based on empirical performance and sensor characteristics:

Table 3: Tunable Stage 1 hyperparameters.

Parameter	**Default**	**Range**	**Governed by**
Hidden dim d	16	8–64	Memory budget vs. capacity
SSM state dim N	16	8–64	Temporal modeling richness
Mamba layers	1	1–3	Temporal depth vs. memory
Spatial kernel k_{PSF}	5	3–7	Sensor PSF FWHM
TDC scales	$\{1, 4, 8\}$	$\{1, 2, 4, 8, 16\}$	Target velocity range
Chunk size P	8192	1024–32768	GPU memory per chunk
Direction bins	8	4–16	Coherence filter resolution

6 Two-Stage Pipeline and Interface

6.1 Pipeline Overview

The complete detection pipeline executes two stages sequentially on the same GPU:

1. **Stage 1** produces the heatmap $\mathcal{H} \in \mathbb{R}^{9\times1024\times1024}$ from the full input volume V.
2. **Post-processing** extracts candidate patches from $\mathcal{H}$.
3. **Stage 2** performs precise segmentation on each candidate patch independently.
4. **Merge** reassembles per-patch segmentation masks into the full-resolution output.

Stage 1 memory is fully released before Stage 2 begins, so peak pipeline memory is the maximum of the two stages, not their sum.

6.2 Post-Processing: From Heatmap to Candidate Patches

Post-processing converts the continuous heatmap $\mathcal{H}$ into a discrete set of candidate patch locations. It proceeds in three steps.

6.2.1 Detection Thresholding

Apply a low threshold ϵ_{det} to the detection channel:

$$M_{\text{det}}(\mathbf{p}) = \begin{cases} 1 & \text{if } \mathcal{H}_{\text{det}}(\mathbf{p}) > \epsilon_{\text{det}}, \\ 0 & \text{otherwise.} \end{cases} \tag{51}$$

The threshold ϵ_{det} is set aggressively low (e.g., $\epsilon_{\text{det}} = 0.1$) to maximize recall. Precision is addressed in the coherence filtering step.

6.2.2 Motion Coherence Filtering

For each pixel where $M_{\text{det}}(\mathbf{p}) = 1$, compute the local coherence score $\Gamma(\mathbf{p})$ from eq. (14) using the predicted direction channel $\mathcal{H}_{\text{dir}}$. The dominant direction bin at each pixel is $c^*(\mathbf{p}) = \arg\max_c \mathcal{H}_{\text{dir}}(c, \mathbf{p})$, mapped to an angle $\hat{\theta}(\mathbf{p}) = c^*(\mathbf{p}) \cdot \frac{2\pi}{8}$.

Pixels with high coherence ($\Gamma(\mathbf{p}) > \gamma_{\text{coh}}$, e.g., $\gamma_{\text{coh}} = 0.85$) in a large neighborhood (e.g., $\mathcal{N}(\mathbf{p})$ is a 64×64 window) are reclassified as background:

$$M_{\text{filtered}}(\mathbf{p}) = M_{\text{det}}(\mathbf{p}) \cdot \mathbb{1}[\Gamma(\mathbf{p}) < \gamma_{\text{coh}}]\,. \tag{52}$$

This step eliminates large swaths of cloud-triggered false positives, since cloud regions produce spatially uniform direction predictions over hundreds of pixels. True targets, occupying only a PSF-sized neighborhood, have direction predictions that are incoherent with their 64×64 surroundings.

6.2.3 Connected Component Extraction and Patch Generation

Connected component labeling is applied to M_{filtered} (using 8-connectivity). For each connected component j:

1. Compute the detection-weighted centroid: $\bar{\mathbf{p}}_j = \frac{\sum_{\mathbf{p} \in j} \mathcal{H}_{\text{det}}(\mathbf{p}) \cdot \mathbf{p}}{\sum_{\mathbf{p} \in j} \mathcal{H}_{\text{det}}(\mathbf{p})}$.
2. Compute the component score: $s_j = \max_{\mathbf{p} \in j} \mathcal{H}_{\text{det}}(\mathbf{p})$.
3. If the component area exceeds a maximum size threshold (e.g., >2000 pixels), discard it as likely cloud leakage. This threshold should exceed the largest expected target footprint; for a target at maximum velocity $|\mathbf{v}_k| = 3$ px/frame over $T = 64$ frames with PSF width $w_{\text{PSF}} = 5$ pixels, the trajectory footprint is approximately $3 \times 64 \times 5 = 960$ pixels, so 2000 provides adequate margin.

Components are ranked by score s_j in descending order. For each retained component, a $128 \times 128 \times T$ patch is extracted from the original volume V, centered on $\bar{\mathbf{p}}_j$ (clamped to ensure the patch lies within the 1024×1024 grid).

6.3 Patch Overlap and Boundary Handling

Two adjacent components whose centroids are separated by less than $128 - 2\delta$ pixels (where $\delta = 64$ is the overlap margin) would produce patches that share substantial spatial area. To avoid redundant Stage 2 computation:

1. If two component centroids $\bar{\mathbf{p}}_i$, $\bar{\mathbf{p}}_j$ are separated by less than 64 pixels, merge them: assign a new centroid at the detection-weighted mean of the union, extract a single 128×128 patch.
2. If separation is between 64 and 128 pixels, extract both patches with overlap. After Stage 2, merge the overlapping segmentation masks via score-based selection: for each pixel in the overlap region, retain the prediction from the patch whose detection score s_j is higher.

3. If separation exceeds 128 pixels, patches are disjoint and processed independently.

This strategy ensures that targets near patch boundaries are always contained within at least one patch with sufficient spatial margin for Stage 2 to make a confident prediction.

6.4 Stage 2 Interface Contract

Stage 2 is specified here only at its interface level; its internal architecture (3D convolutional U-Net with 3D Swin Transformer bottleneck) is documented separately and is treated as a validated black box.

Input. Each candidate patch is a volume $V_{\text{patch}} \in \mathbb{R}^{1\times1\times128\times128\times T}$ extracted from the original full-resolution volume V at the location determined by Stage 1 post-processing. No preprocessing, normalization, or feature extraction is applied—Stage 2 receives raw radiance values identical to the corresponding region of the original input.

Output. For each input patch, Stage 2 produces a confidence score $c \in [0, 1]$ and a binary segmentation mask $M_{\text{seg}} \in \{0, 1\}^{128\times128\times T}$ indicating target presence at each pixel and frame.

Execution. Candidate patches are processed sequentially (batch size 1) to stay within GPU memory. Stage 1 tensors are fully deallocated before Stage 2 begins. For a typical scene yielding 5–15 candidate patches, Stage 2 wall-clock time scales linearly with the number of candidates.

6.5 Output Assembly

The per-patch segmentation masks are placed back into the full $1024 \times 1024 \times T$ coordinate frame at their respective extraction locations. For overlapping regions, the mask from the patch with higher confidence c takes precedence. The final output is:

$$M_{\text{final}} \in \{0, 1\}^{1024\times1024\times T}, \tag{53}$$

a full-resolution spatiotemporal segmentation mask indicating target presence at every pixel and frame.

6.6 Evaluation Metrics

Stage 1 standalone. The primary metric is *recall at K patches* (recall@K): the fraction of ground-truth targets whose trajectory centroid falls within at least one of the top-K candidate patches ranked by score s_j. Targets:

- recall@10 ≥ 0.95,
- recall@20 ≥ 0.99.

The secondary metric is *false-positive patch rate*: the average number of candidate patches on scenes with zero targets. Target: < 15 patches on zero-target scenes.

Full pipeline. Standard detection metrics (precision, recall, F1) computed at the target level: a target is considered detected if any pixel in its ground-truth trajectory is marked positive in M_{final}. Segmentation quality is measured by per-target temporal IoU between the predicted and ground-truth masks.

7 Justification and Alternatives Analysis

This section evaluates the proposed Stage 1 architecture against the principal alternatives considered during the design process. For each alternative, we analyze its theoretical suitability, identify the specific constraint it violates, and explain why the proposed architecture avoids that failure mode.

7.1 Decision Framework

The Stage 1 architecture must simultaneously satisfy five constraints from section 2.4: spatial resolution preservation, temporal resolution preservation, memory budget, exploitation of target sparsity, and generalizability. Each alternative architecture violates at least one of these constraints in a manner that is fundamental rather than tunable—i.e., the violation cannot be resolved by adjusting hyperparameters within the architecture's framework.

The decision tree that led to the proposed architecture proceeded as follows:

1. The no-spatial-downsampling constraint eliminates all architectures that rely on spatial striding or pooling (standard U-Nets, feature pyramid networks, most object detectors).
2. The memory constraint at 1024^2 spatial resolution eliminates dense 3D convolutions and spatial self-attention.
3. The joint spatiotemporal coupling requirement eliminates purely factored (2+1)D approaches and per-pixel temporal-only models without spatial mixing.
4. The remaining viable design space is: per-pixel temporal modeling with deferred spatial reentanglement. Within this space, SSMs (Mamba) dominate convolutions and attention on the temporal axis due to linear memory scaling.

7.2 Alternative 1: Dense 3D CNN at Full Resolution

Description. Apply the same architectural paradigm as the validated Stage 2 model (3D convolutional U-Net) directly to the $1024 \times 1024 \times 64$ volume, using narrow channel widths (e.g., 4→8→16) to control memory.

Why it fails. A 3D convolution with kernel $k_s \times k_s \times k_t$ at full spatial resolution produces intermediate activations of size $C \times H \times W \times T$ at every layer. Even at the narrowest practical width ($C = 4$), a single activation tensor occupies $4 \times 1024^2 \times 64 \times 2 = 0.54\,\text{GB}$ at bfloat16. A network with L layers stores L such tensors for backpropagation (or $\sqrt{L}$ with checkpointing). For a 10-layer encoder-decoder, this yields 5–10 GB in activations alone—comparable to the proposed architecture, but with a critical difference: the 3D convolution also requires $O(C^2 \cdot k_s^2 \cdot k_t)$ multiply-accumulate operations per spatial location per layer, and the *computational cost* (not just memory) scales quadratically with C and cubically with the kernel size.

More fundamentally, the temporal receptive field of a 3D CNN with kernel size $k_t = 3$ requires $L \geq 32$ layers to span the full $T = 64$ temporal window. At 32+ layers with even $C = 4$, the network becomes extremely deep and narrow—difficult to train (vanishing gradients in the temporal direction) and slow to converge. Increasing k_t to reduce depth increases per-layer memory proportionally.

The proposed architecture avoids this by using Mamba's recurrent structure to span the full temporal window in a single pass with $O(N)$ state memory per pixel, independent of T.

Quantitative comparison. A 10-layer 3D CNN with $C = 8$, $k_s = 3$, $k_t = 3$ at $1024^2 \times 64$: each activation tensor is 1.07 GB at bfloat16; with gradient checkpointing, peak activation memory during backpropagation is ∼10 GB (checkpoint storage plus recomputed segment activations and gradients), with additional overhead from convolution workspace buffers that varies by implementation. The temporal receptive field is only 21 frames—insufficient for the full 64-frame window without temporal striding or pooling that violates the temporal preservation constraint. Increasing C to expand representational capacity (e.g., $C = 16$–32) scales activation memory proportionally, pushing estimates to 20–35 GB. The proposed architecture: ∼12 GB with full 64-frame temporal context.

7.3 Alternative 2: (2+1)D Factored Convolutions

Description. Decompose 3D convolutions into 2D spatial convolutions followed by 1D temporal convolutions, as in R(2+1)D or S3D architectures. This reduces parameter count and memory relative to dense 3D convolutions.

Why it fails. As formalized in section 4.3.1, the 1D temporal convolution at a fixed spatial location (eq. (16)) cannot discriminate motion direction. A brightness transient at pixel $\mathbf{p}$ produced by a rightward-moving target is identical, in the temporal domain alone, to one produced by a leftward-moving target, a moving cloud edge, or a noise spike.

This limitation is not merely theoretical. The validated Stage 2 model was tested with (2+1)D factored convolutions during development, and it failed to achieve adequate detection performance on the $128 \times 128 \times 64$ patches where the full 3D model succeeds. The failure was specifically attributed to the inability to discriminate target motion from cloud motion when spatial appearance is identical—precisely the regime described in section 4.2.2.

The proposed architecture addresses this by separating the temporal and spatial stages *with a specific re-entanglement step*: the Mamba backbone extracts rich per-pixel temporal features, and the depthwise separable spatial mixer then correlates these features across the PSF neighborhood. Unlike (2+1)D, the spatial mixer operates on *temporal feature vectors* rather than raw brightness, so it can detect the spatial correlation patterns that encode motion direction.

7.4 Alternative 3: Temporal Shift Module (TSM)

Description. TSM shifts a fraction of feature map channels along the temporal axis by ±1 frame, allowing standard 2D CNNs to achieve pseudo-3D receptive fields at the computational cost of 2D convolutions.

Why it fails. TSM shifts discrete channel slices by exactly ±1 frame. For a subpixel target moving at 0.3 px/frame, the brightness redistribution between adjacent frames is a fractional-pixel PSF shift—an extremely subtle signal. The ±1 frame shift provides a crude first-order temporal difference, but it cannot capture the multi-frame trajectory structure that distinguishes target motion from jitter (which also produces frame-to-frame shifts).

TSM is effective for action recognition in video, where temporal changes are large, spatially distributed, and semantically rich. In the subpixel detection regime, the temporal changes are sub-noise, spatially confined to the PSF footprint, and purely kinematic. The ±1 shift is too coarse a temporal tool for this signal regime.

Furthermore, TSM provides no mechanism for long-range temporal integration: each layer sees only ±1 frame of additional context. Spanning the full 64-frame window requires stacking many TSM layers, re-introducing the depth problem of the 3D CNN alternative.

The proposed architecture uses Mamba's recurrent scan to integrate the full temporal window in one pass, with continuous (not discrete) temporal sensitivity governed by the learned SSM dynamics.

7.5 Alternative 4: Spatial Self-Attention (Vision Transformer)

Description. Apply a Vision Transformer (ViT) or Swin Transformer to each frame independently, then aggregate temporal features. Alternatively, apply spatiotemporal attention (Video Swin, TimeSformer) to the full volume.

Why it fails. Spatial self-attention at 1024×1024 resolution is computationally prohibitive. Even windowed attention (Swin) with window size $w = 7$ requires storing attention maps totaling $\frac{H \cdot W}{w^2} \times w^2 \times w^2 \times n_h \times 2$ bytes at bfloat16, where n_h is the number of attention heads. For $n_h = 4$: $1024^2 \times 49 \times 4 \times 2 \approx 0.41\,\text{GB}$ per layer in attention maps alone. A 12-layer transformer yields $\sim$5 GB for attention storage before accounting for MLP blocks, layer norms, and gradients.

More critically, spatial attention is unnecessary for this problem. The target has no spatial structure at the sensor resolution (section 4.2.2); spatial relationships beyond the PSF neighborhood (3–5 pixels) carry no discriminative information for target detection. Global or windowed spatial attention wastes capacity modeling long-range spatial dependencies (cloud texture, terrain structure) that are irrelevant to the detection task.

The proposed architecture restricts spatial interaction to a single 5×5 depthwise convolution—precisely the PSF-scale neighborhood where spatial information is informative—at a fraction of the cost.

Spatiotemporal attention (Video Swin, TimeSformer) combines the worst of both scaling problems: $O(T^2)$ temporal and $O(w^2)$ spatial, yielding memory that is doubly prohibitive at $1024^2 \times 64$.

7.6 Alternative 5: Per-Pixel Temporal Models Without Spatial Mixing

Description. Process each pixel's temporal sequence independently through a 1D CNN, LSTM, or SSM, producing a per-pixel feature vector, and classify each pixel independently without any spatial interaction.

Why it fails. This is the "pure per-pixel Mamba" approach without the depthwise separable spatial mixer. Note that the proposed architecture also begins with per-pixel temporal processing (Mamba); the critical difference is what happens afterward. Without spatial mixing, a single pixel's temporal sequence is insufficient for motion direction discrimination. The failure mode is related to but more specific than the (2+1)D case (section 7.3): a subpixel target distributes energy across a PSF-sized neighborhood. As the target moves, the energy *shifts* between pixels in this neighborhood. At any single pixel, this appears as a brightness transient—indistinguishable from cloud-edge motion, jitter, or noise. Only by correlating the temporal features of neighboring pixels (pixel $\mathbf{p}$ brightens while pixel $\mathbf{p} + \Delta\mathbf{p}$ dims, consistent with rightward motion) can the network recover direction.

The proposed architecture includes the spatial mixer (section 5.6) specifically to restore this inter-pixel correlation. The depthwise convolution aggregates temporal features across the PSF footprint, and the pointwise convolution combines them into joint spatiotemporal descriptors. This

is the minimal spatial interaction needed for direction discrimination—more efficient than 3D convolutions or attention, but sufficient for the physics of the problem.

7.7 Alternative 6: Brute-Force Tiling with Stage 2

Description. Bypass Stage 1 entirely. Tile the 1024×1024 scene into overlapping 128×128 patches and run Stage 2 on every patch. With stride 64, this yields $(\lfloor (1024 - 128)/64 \rfloor + 1)^2 = 15^2 = 225$ patches.

Why it fails. Each Stage 2 inference on a $128 \times 128 \times 64$ patch requires 35–65 GB and takes t_2 seconds. Processing 225 patches sequentially requires $225 \times t_2$ wall-clock time. The proposed pipeline processes 5–15 patches plus the lightweight Stage 1 pass (which is substantially faster than a single Stage 2 invocation), yielding a 15–45× reduction in Stage 2 compute.

At stride 128 (no overlap), the count drops to 64 patches but targets near patch boundaries are missed. Even at stride 64, a target whose trajectory crosses a boundary may be split across two patches, each containing insufficient temporal context for detection.

The proposed two-stage pipeline reduces the number of Stage 2 invocations by 15–45× while maintaining or improving recall, because Stage 1's coherence filtering eliminates the vast majority of background patches that brute-force tiling would wastefully process.

Table 4: Brute-force tiling vs. proposed two-stage pipeline.

	Brute-force (stride 64)	**Proposed**
Stage 2 invocations	225	5–15
Stage 2 compute (relative)	1×	0.02–0.07×
Boundary miss risk	Low (with overlap)	Low (centroid-centered)
Peak GPU memory	35–65 GB	$\max(12, 35\text{–}65) \approx 35\text{–}65$ GB

Quantitative comparison. Memory is comparable (dominated by Stage 2 in both cases), but computational cost is dramatically lower with the proposed pipeline.

7.8 Alternative 7: Classical Detect-Before-Track

Description. Use classical signal processing for Stage 1: frame registration, temporal median subtraction, or CFAR thresholding on temporal statistics, followed by the learned Stage 2 on candidate regions.

Why it fails. Classical approaches require explicit parametric models of the background (number of motion layers, noise distribution, motion model) and hand-tuned thresholds (CFAR parameters, registration tolerances). These parameters must be adjusted when the sensor, platform, or scene characteristics change, violating the generalizability constraint.

Specific failure modes are detailed in section 4.2.4. The most critical is multi-layer cloud handling: classical registration can compensate for a single global motion (terrain), but cannot simultaneously handle 2–4 independent cloud layers moving in different directions without prior knowledge of the number of layers and their velocities. In practice, cloud residuals after single-layer registration dominate the detection statistics, producing false-positive rates that overwhelm any downstream classifier.

The proposed learned Stage 1 implicitly estimates and compensates for the multi-layer motion field through its temporal modeling, without requiring explicit layer estimation or parameterization.

7.9 Alternative 8: Recurrent Feature Refinement (RFR / Deformable Alignment)

Description. Use deformable convolutions or spatial transformer networks to explicitly align features across frames before detection, as in recurrent feature refinement pipelines developed for overhead video.

Why it partially succeeds but is suboptimal for Stage 1. Unlike the previous alternatives, deformable alignment is not fundamentally incompatible with the problem. It can handle multi-layer motion and sensor jitter by learning per-pixel displacement fields. However, it introduces two drawbacks for the Stage 1 role:

First, deformable alignment *compensates* motion—it warps features to a common reference frame, eliminating motion as a signal. But for Stage 1, motion is the *discriminative* signal: we want to *detect* anomalous motion, not remove it. Alignment-based approaches are better suited for Stage 2, where the goal is to precisely segment a target within a small patch where the target's presence is already suspected.

Second, deformable convolutions are memory-intensive. Each deformable conv layer stores per-pixel offset fields ($2 \times k^2 \times H \times W$ per layer), which at 1024^2 resolution adds significant overhead. For a 6-layer deformable alignment module: $6 \times 2 \times 9 \times 1024^2 \times 4 \approx 0.45\,\text{GB}$ in offsets alone, plus the attention masks and aligned feature maps.

The proposed architecture achieves motion detection (not compensation) more efficiently via the Mamba temporal encoder, which learns to flag temporal anomalies without explicitly estimating displacement fields.

7.10 Alternative 9: Sparse 3D Convolutions

Description. Use spatially sparse 3D convolutions (e.g., via MinkowskiEngine or TorchSparse) that only compute at spatial locations with non-zero activations, exploiting the sparsity of the target signal.

Why it is premature for Stage 1. Sparse convolutions are powerful when the input itself is sparse (e.g., point clouds, voxelized 3D scenes). However, the Stage 1 input is *dense*: every pixel has a non-zero radiance value, and the target signal is indistinguishable from background in any single frame. There is no natural sparsification of the input that preserves the faint target signatures.

Sparsity could be introduced *after* an initial dense pass identifies candidate locations—but this is precisely the two-stage strategy already proposed, with Stage 1 serving as the dense pass and Stage 2 operating on the sparse candidates.

Applying sparse convolutions within Stage 1 after the first few dense layers (sparsifying based on early activations) is a viable optimization but carries risk: aggressive sparsification may discard pixels where the faint target signal has not yet been amplified above the sparsification threshold. This is a tunable implementation optimization, not an architectural alternative; it could be applied to the proposed architecture if empirical results show that early layers reliably activate at target locations.

7.11 Summary of Alternatives

Table 5: Alternative architectures and their primary failure modes.

Alternative	Primary violation	Severity
Dense 3D CNN	Memory + limited temporal RF	Fundamental
(2+1)D factored	No direction discrimination	Fundamental
TSM	Coarse temporal, no long-range	Fundamental
Spatial attention (ViT/Swin)	Memory at 1024^2	Fundamental
Pure temporal (no spatial)	No direction discrimination	Fundamental
Brute-force tiling	15–45× compute cost	Practical
Classical DBT	Generalizability	Requirement
Deformable alignment	Removes (vs. detects) motion	Suboptimal
Sparse 3D conv	Input is dense, premature	Suboptimal

The proposed bidirectional Mamba backbone with deferred depthwise separable spatial mixing is the architecture that satisfies all five constraints simultaneously: full spatial resolution, full temporal context, tractable memory, exploitation of target sparsity (via the two-stage pipeline), and full generalizability (no hand-tuned parameters).

8 Training Requirements

8.1 Synthetic Data Requirements

Stage 1 training relies primarily on synthetic data with known ground-truth target locations, velocities, and occlusion states. This section specifies what the model requires from the synthetic data generator, not the generator's internal implementation.

8.1.1 Scene Composition

Each training volume $V \in \mathbb{R}^{H' \times W' \times T}$ (where $H' \times W'$ may be smaller than 1024×1024 during prototyping) must contain:

Background. A radiometrically plausible background comprising terrain and 0–4 independent cloud layers. Each cloud layer must translate at a distinct velocity $\mathbf{v}_{\text{cloud}}^{(l)}$ throughout the temporal window, with spatially varying optical depth $\tau^{(l)}(\mathbf{p}, t) \in [0.0, 3.0]$. The terrain must exhibit apparent motion consistent with a platform velocity $\mathbf{v}_{\text{plat}}$. Cloud texture must be visually indistinguishable from terrain texture in any single frame—this is the defining characteristic of the overhead surveillance clutter environment and must be faithfully reproduced.

Targets. Each volume contains $K \in \{0, \ldots, 20\}$ targets. Approximately 30% of training volumes should contain zero targets, to train the network to produce low false-positive rates on empty scenes. Each target k is specified by:

- Initial subpixel position $\mathbf{p}_k(0) \in \mathbb{R}^2$, uniformly distributed across the spatial extent.
- Velocity $\mathbf{v}_k$ with speed $|\mathbf{v}_k| \in [0.2, 3.0]$ px/frame and direction $\theta_k \in [0, 2\pi)$ uniformly distributed.

- Intrinsic radiance L_k yielding $\mathrm{SNR}_{\mathrm{frame}} \in [0.3, 2.0]$ at the peak pixel, spanning the faint-to-moderate detection regime.
- Spatial profile: a point source convolved with the sensor PSF $h(\mathbf{p})$ of known FWHM w_{PSF}.

Sensor effects.

- PSF convolution applied to all injected targets before adding to the background.
- Per-frame jitter: a global 2D shift $\boldsymbol{\delta}(t) \sim \mathcal{N}(\mathbf{0}, \sigma^2_{\mathrm{jitter}}\mathbf{I})$ with $\sigma_{\mathrm{jitter}} \in [0.05, 0.3]$ px, applied after scene composition. The training range extends below the nominal operational range ($\sigma_{\mathrm{jitter}} \in [0.1, 0.3]$ px from section 2.2.2) to expose the SE gating mechanism to near-jitter-free conditions and improve generalization to stabilized sensors.
- Additive sensor noise calibrated to the real sensor's noise floor.

8.1.2 Occlusion Modeling

Occlusion must be modeled as continuous attenuation (eq. (4)), not binary masking. For each target, the generator must compute the total cloud optical depth along the line of sight at each frame, producing a time-varying transmission curve $\mathcal{T}_{\mathrm{total}}(\mathbf{p}_k(t), t)$.

Training data must include:

- Fully visible targets ($\mathcal{T}_{\mathrm{total}} \approx 1$ for all t): $\sim$30% of targets.
- Partially attenuated targets ($\mathcal{T}_{\mathrm{total}} \in [0.3, 0.8]$ for some portion of the window): $\sim$30%.
- Severely occluded targets ($\mathcal{T}_{\mathrm{total}} < 0.1$ for 30–40 consecutive frames, i.e., 50–60% of the temporal window): $\sim$30%.
- Permanently occluded targets ($\mathcal{T}_{\mathrm{total}} < 0.1$ for $> 75\%$ of the window): $\sim$10%. These are expected false negatives; their inclusion prevents the network from hallucinating targets in persistently cloudy regions.

The gradual-fade transition as a target enters/exits a cloud edge is particularly important. The transition zone should span 5–20 frames depending on the cloud's spatial gradient and the target's velocity relative to the cloud edge.

8.1.3 Ground-Truth Labels

For each training volume, the following ground-truth annotations are required:

Detection mask. $G_{\mathrm{det}} \in \{0, 1\}^{H' \times W'}$: a binary spatial mask where a pixel is labeled positive if the target's trajectory centroid passes within $\lceil w_{\mathrm{PSF}}/2 \rceil$ pixels of that location during any frame in the temporal window. This dilated labeling accounts for the PSF-distributed energy footprint.

Direction class map. $G_{\mathrm{dir}} \in \{0, 1, \ldots, 7\}^{H' \times W'}$: at each positive pixel (where $G_{\mathrm{det}} = 1$), the direction bin is $c^* = \lfloor \theta_k / (2\pi/8) \rfloor \in \{0, \ldots, 7\}$, corresponding to the 8 channels of $\mathcal{H}_{\mathrm{dir}}$. The value of G_{dir} at background pixels is arbitrary, since the direction loss is computed only over the positive set $\mathcal{P}$ (eq. (56)).

Occlusion fraction. $G_{\mathrm{occ}} \in [0, 1]^K$: per-target fraction of frames with $\mathcal{T}_{\mathrm{total}} < 0.1$. Used for stratified evaluation (measuring recall as a function of occlusion severity), not as a training target.

8.2 Loss Formulation

The total training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \lambda_{\text{dir}} \cdot \mathcal{L}_{\text{dir}}, \tag{54}$$

where λ_{dir} is a weighting coefficient (default $\lambda_{\text{dir}} = 0.5$).

Detection loss. Focal loss (eq. (33)) applied to the detection channel $\mathcal{H}_{\text{det}}$ against the ground-truth mask G_{det}:

$$\mathcal{L}_{\text{det}} = \frac{1}{H'W'} \sum_{\mathbf{p}} \mathcal{L}_{\text{focal}}(\mathcal{H}_{\text{det}}(\mathbf{p}),\ G_{\text{det}}(\mathbf{p})), \tag{55}$$

with $\gamma = 2$ and class-balancing weight $\alpha = 0.75$ for positives ($\alpha = 0.25$ for negatives).

Direction loss. Categorical cross-entropy applied only at positive pixels, with stop-gradient on the backbone features:

$$\mathcal{L}_{\text{dir}} = -\frac{1}{|\mathcal{P}|} \sum_{\mathbf{p} \in \mathcal{P}} \log \mathcal{H}_{\text{dir}}(G_{\text{dir}}(\mathbf{p}),\ \mathbf{p}), \tag{56}$$

where $\mathcal{P} = \{\mathbf{p} : G_{\text{det}}(\mathbf{p}) = 1\}$ is the set of positive pixels. The stop-gradient operator on X_4 in the direction head (eq. (49)) ensures that gradients from $\mathcal{L}_{\text{dir}}$ do not propagate into the backbone. This decoupling prevents early-stage noisy direction predictions from corrupting the detection features.

Gradient flow. $\mathcal{L}_{\text{det}}$ backpropagates through the detection head, spatial mixer, temporal aggregation, Mamba backbone, projection, and SE-gated TDC—the entire network. $\mathcal{L}_{\text{dir}}$ backpropagates only through the direction head's 1×1 convolution weights. After a configurable number of warm-up epochs (default: 10), the stop-gradient can optionally be relaxed to allow end-to-end direction supervision, though empirical testing should confirm this improves rather than harms detection recall.

8.3 Training Curriculum

Training proceeds in three phases:

Phase A: Stage 1 standalone (primary). Train Stage 1 in isolation on synthetic data at the prototype spatial resolution (e.g., $256 \times 256 \times 64$). Objective: maximize recall@10 ≥ 0.95 and recall@20 ≥ 0.99 on a held-out validation set with representative occlusion and cloud diversity. Use the Adam optimizer with learning rate 1×10^{-3}, cosine annealing, and gradient checkpointing on the Mamba backbone. Train for 50–100 epochs; convergence is expected within 30 epochs given the small parameter count ($\sim$7,000 parameters).

Phase B: Scale to full resolution. Once Phase A validates the architecture on tiles, scale to $1024 \times 1024 \times 64$ inputs. Because the architecture is fully convolutional (no learned position embeddings or fixed-size layers), the weights from Phase A transfer directly—no architectural changes are needed. Fine-tune for 10–20 epochs at full resolution with a reduced learning rate (1×10^{-4}) to adapt to the full-scene statistics (larger cloud structures, more diverse motion fields, more targets per scene).

Phase C: Stage 2 hard-negative mining (optional). Freeze Stage 1. Run it on the full training set to generate candidate patches. Collect false-positive patches (candidates that contain no target) and true-positive patches. Add the false-positive patches to Stage 2's training set as hard negatives. This improves Stage 2's precision on the specific false-positive distribution that Stage 1 produces, tightening the overall pipeline performance. Stage 2's architecture and training procedure are otherwise unchanged from its existing validated configuration.

8.4 Data Augmentation

Standard spatial augmentations (random flips, 90° rotations) are applied to the full $H' \times W' \times T$ volume, with corresponding transformations to the ground-truth masks and direction labels (direction bins are rotated accordingly).

Temporal augmentations:

- Temporal reversal: flip the volume along t, with direction labels rotated by 180° (bin $c \to (c+4) \bmod 8$).
- Jitter magnitude scaling: multiply σ_{jitter} by a random factor $\in [0.5, 2.0]$ to expose the SE gate to varying noise conditions.
- Target SNR perturbation: multiply each target's radiance L_k by a random factor $\in [0.7, 1.3]$ to diversify the difficulty distribution.

No temporal subsampling or frame dropping is applied, as this would alter the target velocity in pixel-per-frame units, breaking the correspondence between the velocity and the direction label.

9 Operational Considerations

9.1 Full Pipeline Memory Budget

The two stages execute sequentially, so peak memory is the maximum of the two stages.

Table 6: Full pipeline memory budget. Stages execute sequentially; peak is the maximum.

Component	Training peak	Inference peak	Notes
Stage 1 (complete)	12 GB	6 GB	See table 2
Stage 1 post-processing	<0.1 GB	<0.1 GB	CPU-bound
Stage 2 per patch	35–65 GB	20–35 GB	Existing model
Pipeline peak	**35–65 GB**	**20–35 GB**	Stage 2 dominated

Both training and inference peaks fit comfortably within the 192 GB budget. The pipeline is dominated by Stage 2; Stage 1 contributes negligibly to peak memory. This confirms that the proposed Stage 1 architecture achieves its design goal: providing high-recall region proposals at minimal memory cost, concentrating the expensive computation on the small number of candidate patches.

9.2 Inference Throughput

End-to-end inference time for one $1024 \times 1024 \times 64$ volume consists of:

- Stage 1 forward pass: dominated by the 128 sequential Mamba chunk passes. Each chunk ($P = 8192$ sequences of length 64) is a small batch SSM scan—expected ∼5–15 ms per chunk,

or ∼1–2 s total for the full grid. The spatial mixer and output heads add negligible time (<50 ms).

- Post-processing: connected component labeling and coherence filtering on a 1024×1024 binary mask—<100 ms on CPU.
- Stage 2: 5–15 sequential forward passes on $128 \times 128 \times 64$ patches. Each pass takes t_2 seconds (hardware-dependent; typically 1–5 s per patch on an A100/H100). Total: $5t_2$ to $15t_2$ seconds.

Total inference per volume: ∼2 s (Stage 1) + 5–75 s (Stage 2) = 7–77 s, dominated by Stage 2. For applications requiring faster throughput, Stage 2 patches can be batched if memory permits (e.g., 2–3 patches per batch on a 192 GB system), reducing wall-clock time proportionally.

9.3 Hardware Requirements

Minimum viable configuration. A single GPU with ≥40 GB VRAM (e.g., NVIDIA A100 40 GB) is sufficient for Stage 1 training at full $1024 \times 1024 \times 64$ resolution. Stage 2 inference on candidate patches may also fit on 40 GB depending on the model's exact memory footprint, but headroom is marginal at the upper end of the 20–35 GB inference estimate.

Recommended configuration. A single GPU with ≥80 GB VRAM (e.g., NVIDIA A100 80 GB or H100) accommodates both stages with headroom for larger hidden dimensions ($d = 32$–64) and reduced spatial chunking during Stage 1.

Multi-GPU. The architecture does not require multi-GPU training for Stage 1 (the parameter count is too small to benefit from model parallelism, and the batch size is 1). Multi-GPU can accelerate Stage 2 by distributing candidate patches across devices, but this is an engineering optimization outside the scope of this document.

9.4 Deployment Considerations

Input precision. The input volume must be provided in float32 to preserve subpixel brightness gradients. Quantizing the input to float16 or int16 risks destroying the faint target signatures that lie below the quantization step size. This requirement applies to both Stage 1 and Stage 2 inputs.

Threshold calibration. The detection threshold ϵ_{det} and coherence threshold γ_{coh} are the only tunable parameters at inference time. These should be calibrated on a representative validation set to achieve the desired recall/false-positive tradeoff. A receiver operating characteristic (ROC) curve plotting recall@K vs. K (number of candidate patches) at various ϵ_{det} values provides the calibration tool.

Retraining for new sensors. When the sensor, platform, or target characteristics change, Stage 1 must be retrained on synthetic data generated with the new parameters (PSF width, noise model, GSD, platform velocity). The architecture itself requires no modification—only the data changes. The spatial mixer kernel size k_{PSF} should be adjusted if the PSF FWHM changes substantially (e.g., from $w_{\text{PSF}} = 3$ to $w_{\text{PSF}} = 7$ pixels). Retraining from scratch is expected to converge within 30 epochs given the small parameter count.

Graceful degradation. Under conditions outside the training distribution (e.g., extreme jitter, cloud optical depths beyond the training range, target velocities outside $[0.2, 3.0]$ px/frame), the network's behavior is undefined. The SE gating mechanism provides partial robustness to elevated jitter (by suppressing TDC channels), but systematic robustness requires expanding the training distribution to cover the anticipated operational envelope.

9.5 Prototyping Roadmap

The recommended prototyping sequence, ordered by information value and implementation effort:

Step 1: Validate recall on tiles (∼1 week). Train Stage 1 on $256 \times 256 \times 64$ synthetic tiles. Measure recall@10 and recall@20 on a validation set with realistic cloud and occlusion distributions. Profile peak GPU memory with `torch.cuda.max_memory_allocated()`. If recall@10 < 0.90, increase hidden dimension d or add a second Mamba layer before proceeding.

Step 2: Validate coherence filter (∼2 days). On the trained tile model, measure the candidate patch count on zero-target scenes and multi-cloud scenes. If false-positive patch rate > 15, tune γ_{coh} or increase direction bins from 8 to 12–16.

Step 3: Scale to full resolution (∼1 week). Transfer tile weights to $1024 \times 1024 \times 64$. Fine-tune and confirm that recall and memory estimates hold at full scale. This step validates the spatial chunking implementation and the tiling-free memory hypothesis.

Step 4: End-to-end pipeline (∼1 week). Wire Stage 1 output to Stage 2 via the post-processing and patch extraction pipeline. Measure end-to-end precision, recall, and F1 on a held-out test set. Optionally run Phase C (hard-negative mining) to tighten Stage 2 precision.

Step 5: Operational validation. Test on real sensor data (if available) or high-fidelity physics-based synthetic scenes. Calibrate thresholds for the target operational scenario. Document failure modes and edge cases for the deployment team.